

A Multi-cycle RV32I System on Chip for the Digilent Genesys 2

Final project report

Amit Malach

Supervisor: Prof. Avi Mendelson

Technion – Israel Institute of Technology

November 11, 2026

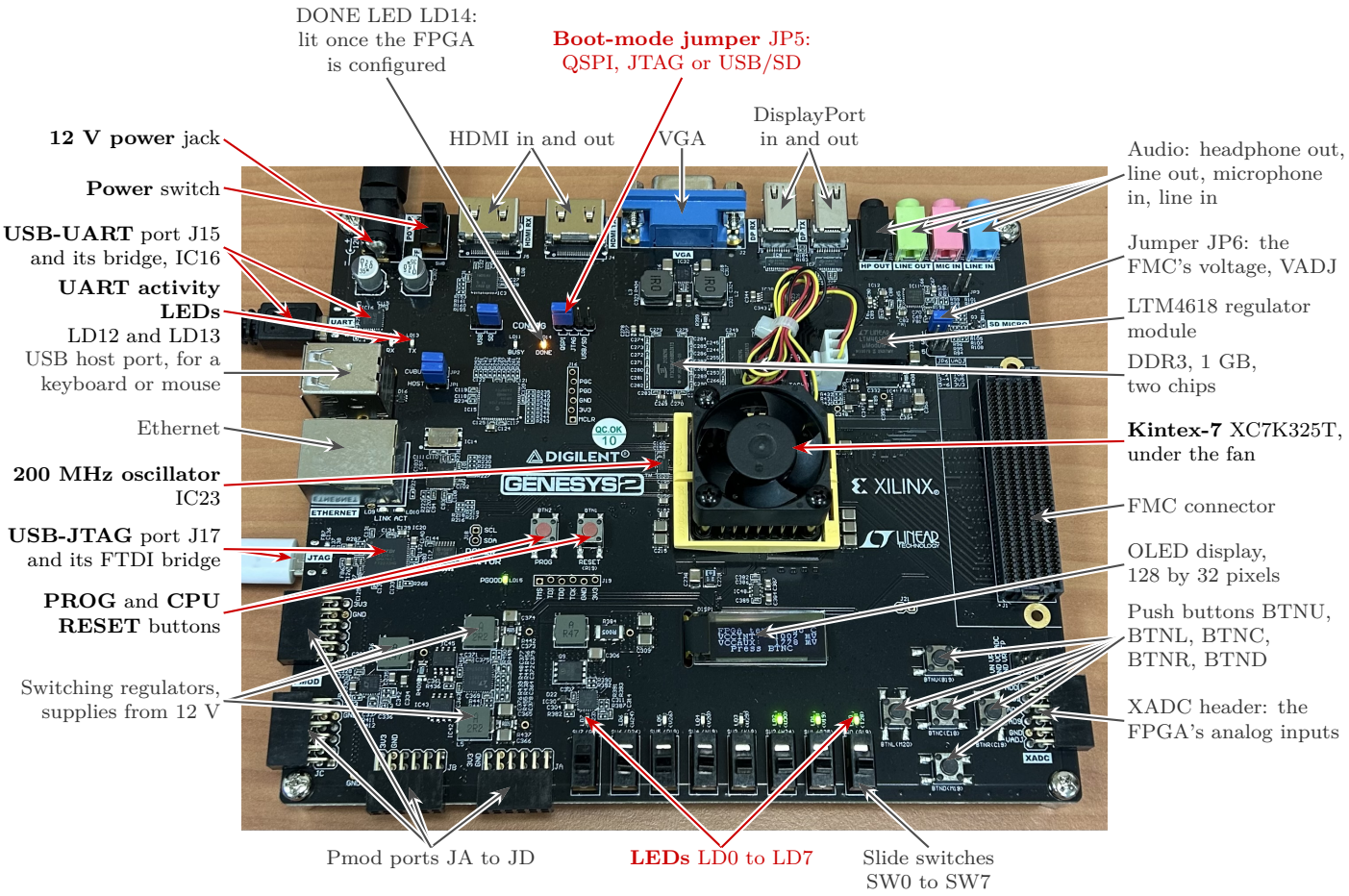


Figure 1: The Digilent Genesys 2 from above, powered on and running Digilent's factory demo. In **red**, the parts this design uses.

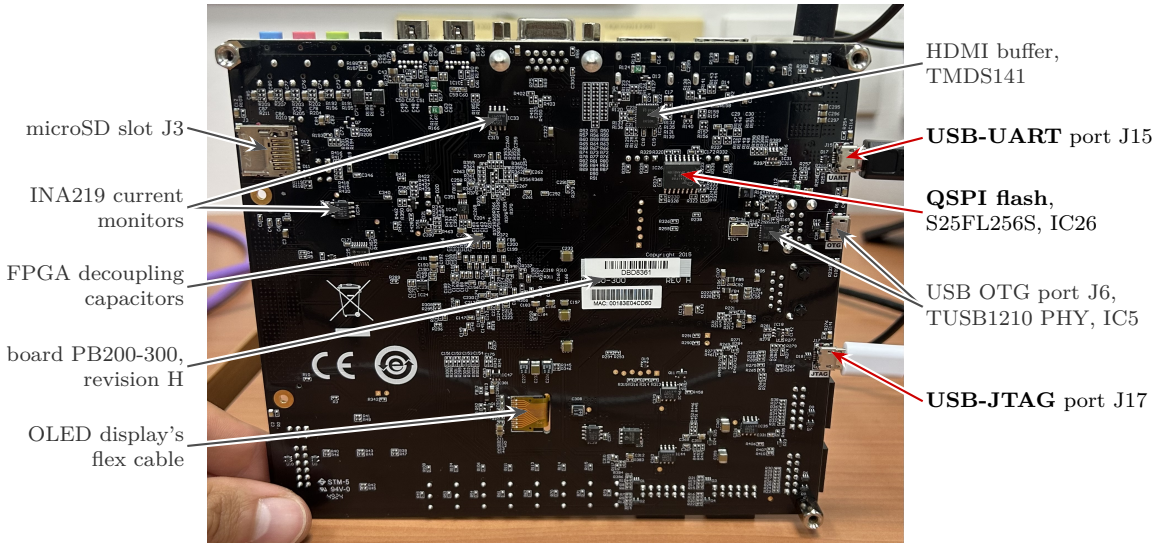


Figure 2: The underside, the top view mirrored. In **red**, the parts this design uses.

Abstract

This report describes a small system on chip built around a multi-cycle RV32I core, written in Verilog and run at 200 MHz on the Kintex-7 FPGA of a Digilent Genesys 2 board. The core executes the RV32I base integer instruction set, one instruction at a time, in four to six clock cycles each, under a seven-state finite state machine. Around it sit 6 KB of block RAM, a memory-mapped LED register, a UART transmitter and a controller for the board's 32 MB QSPI flash, all on one bus that the core sees as plain memory. Two boot modes share the same hardware. In BRAM boot the firmware image is part of the bitstream and the program runs from the RAM. In flash boot the core executes a program in place from the flash, which admits programs larger than the RAM at about a hundredth of the speed.

On the board the design draws the Mandelbrot set and a turning RISC-V logo over the UART from its block RAM, renders tinyraytracer's ray-traced picture, a program five times the RAM's size, in place from the flash, and plays the ST-NICCC polygon demo from scene data in the flash. The design is inspired by Bruno Levy's *From Blinker to RISC-V* series.

Contents

1 Introduction **1**
- 1.1 What the board does . . . 1
- 1.2 Starting point and own work . . . 3
- 1.3 What was built . . . 4
- 1.4 Results in brief . . . 4
- 1.5 How to read this report . . . 5

2 The board **6**

3 Repository tour **9**

4 System architecture **10**
- 4.1 The bus and the modules . . . 10
- 4.2 Memory map and address decode . . . 11
- 4.3 Clock and reset . . . 12

5 The RV32I instruction set **13**
- 5.1 Registers and assembly syntax . . . 13
- 5.2 Instruction formats . . . 13
- 5.3 The instructions . . . 14
- 5.4 The calling convention . . . 17

6 The multi-cycle core **18**
- 6.1 The state machine . . . 18
- 6.2 The datapath . . . 19
- 6.3 One instruction through the states . . . 20

7 Memory and memory-mapped IO **22**
- 7.1 The RAM . . . 22
- 7.2 The LED register and the UART . . . 22
- 7.3 The flash stall . . . 22

8 Boot modes and the SPI flash **24**
- 8.1 The flash layout . . . 24
- 8.2 CCLK and STARTUPE2 . . . 26
- 8.3 Execute-in-place . . . 27

9 Verification **28**
- 9.1 The ALU unit bench . . . 28
- 9.2 The golden trace . . . 28
- 9.3 The regression . . . 29

10 Implementation results **30**
- 10.1 Utilization . . . 30
- 10.2 Timing . . . 30
- 10.3 Power . . . 32
- 10.4 The design on the device . . . 33

11 For the next student **34**

12 Conclusions **35**

Bibliography **36**

1 Introduction

This is an undergraduate project with one aim: to build a processor that runs compiled C on a real FPGA board, and to understand every layer between the C source and the lit LED. The result is a multi-cycle RV32I core, 6 KB of RAM, three memory-mapped IO registers and a flash controller, wired into one top level and run on a Digilent Genesys 2 at the board's 200 MHz clock. This chapter shows what the board does with it, then what was built and how the report is laid out.

1.1 What the board does

Every picture in this section is rendered from bytes the board sent over its UART. From the moment a bitstream starts to load over JTAG, the host records the UART at 115 200 baud, 8N1, with a timestamp on every read, accurate to about 2 ms. The demos draw with ANSI escape sequences, each pixel a coloured terminal cell, so a serial terminal shows them as pictures. The QR code on the right of each figure holds the address of the repository's README, `https://github.com/amitmalach/FPGA_RiscV_MultiCycle#running-the-demos`, where every demo below plays as an animation.

Mandelbrot. The committed firmware image draws the Mandelbrot set from the block RAM, in fixed-point arithmetic with at most 20 iterations a pixel. It sends one frame of 46 by 46 pixels a second, steps the colours by one at each frame, and counts the frames on the LEDs. Figure 1.1 shows the first five of the 23 frames captured, which cover a full cycle of the 21 colours.

The first frame the board sent is byte for byte the frame the simulation of the design produced (the golden trace, Section 9.2) and the output of the same C program run on a PC, so the board, the simulation and the C source agree. It took 1031 ms on the board, and the simulation counted 206.3 million cycles for it, which take 1031 ms at 200 MHz: the board runs the cycles the simulation counts. About 956 ms of the frame is the UART sending its bytes, so the computation itself takes about 75 ms.

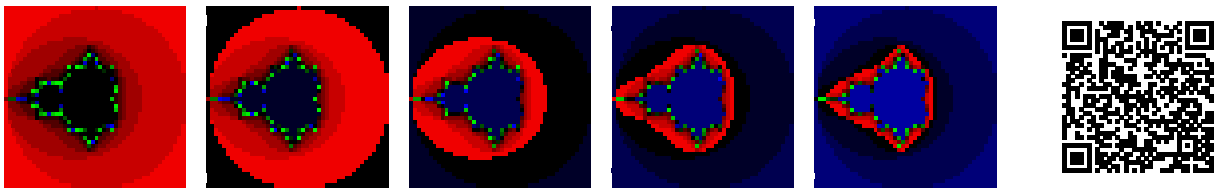


Figure 1.1: The mandelbrot demo on the board, in BRAM boot: the first five of the 23 frames captured, left to right, about a second apart, each pixel two terminal cells on a 24-bit background colour. The animation plays in the repository's README, at `https://github.com/amitmalach/FPGA_RiscV_MultiCycle#running-the-demos`, the address the QR code on the right holds.

riscv_logo. The riscv_logo demo rotates and zooms the RISC-V logo, a 16 by 16 bitmap, over a grid of 40 by 40 pixels in the 16 standard terminal colours. One turn is 64 frames, about half a second each, a pace the UART sets. Figure 1.2 shows every eighth frame. The captured frames are, byte for byte, what `fw/riscv_logo.c` prints when compiled natively.

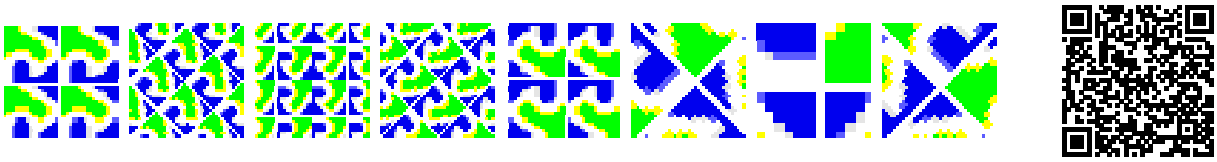


Figure 1.2: The riscv_logo demo on the board, in BRAM boot: frames 0, 8, ..., 56 of one turn, left to right, 45 degrees apart. The whole turn plays in the README, at the address the QR code on the right holds.

tinyraytracer. tinyraytracer is Dmitry Sokolov's small ray tracer [5], a few hundred lines written to teach how a ray-traced picture is made. For each pixel it casts a ray from the eye through the screen, finds the nearest sphere or floor point the ray hits, and colours it from three point lights, with diffuse and specular shading and shadows. Rays that hit the mirror or the glass sphere are reflected or refracted and traced again. Here it prints its picture to the terminal as ANSI colour cells. It is 29 KB of code and constants, five times the RAM, so it runs in flash boot: every instruction is fetched in place from the board's flash, and every floating-point operation is a call into `libgcc`. The board drew the picture row by row in about two minutes, and Figure 1.3 shows it as it grew. Its output is, byte for byte, what `fw/tinyraytracer.c` prints when compiled natively. Two board problems showed on the way: the board printed nothing until the CPU RESET button was pressed after the load (`notes.md`, "Flash boot loses its first fetch"), and writing the image to the flash cost the board its power-on boot until the flash's Quad Enable bit was set again (`notes.md`, "Writing the flash clears the Quad Enable bit").

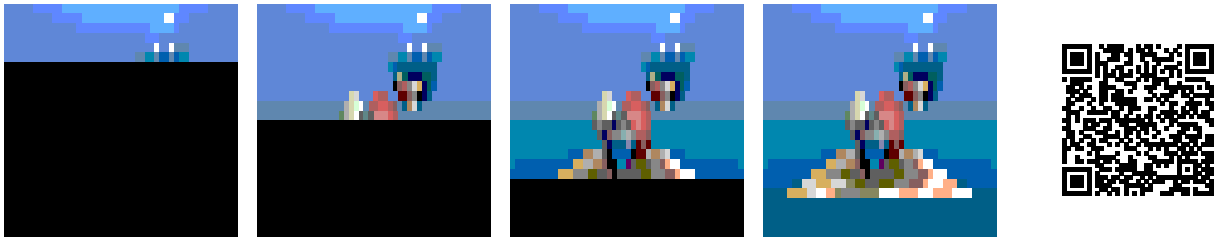


Figure 1.3: tinyraytracer on the board, in flash boot, after 6, 12, 18 and all 24 rows of its picture: 24, 64, 107 and 127 s after its first pixel. Four spheres, of ivory, glass, red rubber and mirror, over a chequered floor and lit by three lights, 48 by 24 terminal cells in xterm's 256 colours. The README plays the drawing, 20 times faster than the board drew it.

sieve. The sieve of Eratosthenes finds the primes below a bound by crossing out the multiples of each prime in turn: whatever is never crossed out is prime. The sieve demo, taken from the PicoRV32 project, sieves the odd numbers from 3 to 129 in a 64-bit bitmap, prints the 31 primes below 128, one a line, and then a checksum of them, `1772A48F`, followed by `OK` when the checksum is the expected one, and starts again after a short delay, a round every 96 ms. Its output is text, so Figure 1.4 shows it as a terminal 12 rows high shows it: once the screen is full, each new line scrolls it up. The captured rounds are, byte for byte, what `fw/sieve.c` prints when compiled natively, but for the checksum's hex digits, which `fw/print.c` prints in upper case.

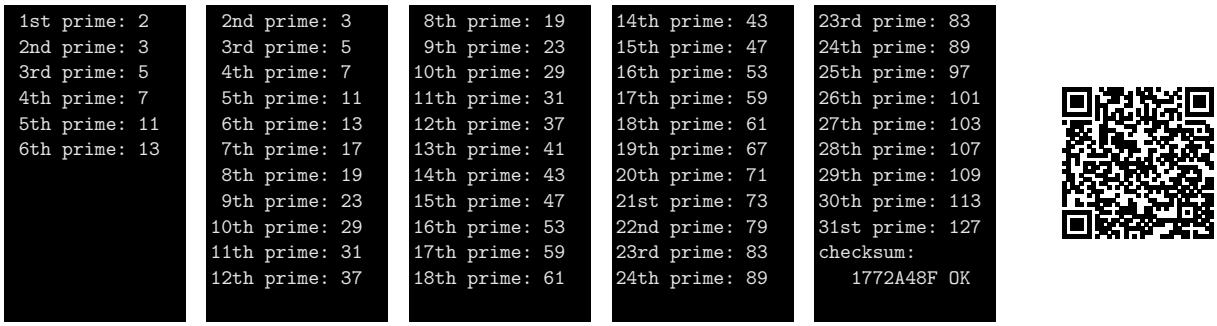


Figure 1.4: The sieve demo on the board, in BRAM boot: a terminal 12 rows high after 6, 12, 18, 24 and 33 lines of the first round, 7, 15, 23, 30 and 42 ms after its first byte. The last line of each screen is where the cursor waits. The README plays the first two rounds, 50 times slower than the board sent them.

ST_NICCC. ST-NICCC 2000 is a demo for the Atari ST by the group Oxygene whose best-known part is a flight through a scene of flat-shaded polygons, streamed from disk. Its scene file was later published for a competition to replay it on other machines [6]. Each frame of the file lists polygons, each with a colour from a palette of 16 and the screen points of its corners. The player, `fw/ST_NICCC.c`, reads the file through the core's flash region and draws each frame as 128 by 50 coloured cells. The file is 640 KB, a hundred times the RAM, so it was written to the flash at `0xC00000`, where Figure 8.1 reserves room for it, and the player runs from the block RAM in BRAM boot. The board drew a frame every 0.9 s, with the UART sending at its full rate. The player never clears the screen: each frame is drawn over the last, so a screen taken in the middle of a frame shows parts of two.

The run is also the clearest evidence that the first read from the flash after configuration is lost (`notes.md`, "Flash boot loses its first fetch"). Its first frame came out wrong, as a wrong first word from the flash makes it: the first SPI transaction after configuration loses clock edges (Section 8.2). The polygons were right again within two seconds, and one palette colour stayed wrong until the scene set it again, 70 s in. From there on, the output is byte for byte what `fw/ST_NICCC.c` prints when compiled natively with the same scene file.

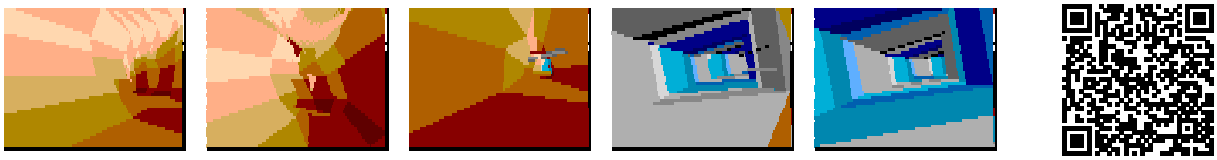


Figure 1.5: ST_NICCC on the board, in BRAM boot, with its scene data in the flash: the screen 76, 97, 118, 139 and 145 s after the player's first byte, 128 by 50 terminal cells in xterm's 256 colours. The README plays the screen every half second, in real time, from 70 s to the end of the capture.

1.2 Starting point and own work

The project started from Bruno Levy's *learn-fpga* series *From Blinker to RISC-V* [1], which grows a multi-cycle RV32I core in small increments, through memory-mapped IO and the GNU toolchain to execution from SPI flash, for small boards such as the IceStick and the ULX3S. The series supplied the starting point: the shape of the state machine, the bus and the demo programs. Taking that starting point to a working, verified system on the Genesys 2, a board the series does not support, was the project's own work, and most of the effort went there.

- **The port to the Genesys 2.** The differential 200 MHz clock, the pin constraints, the

board's UART pin names (`notes.md`, "UART port names and the pin-direction trap"), and the flash's clock, which a 7-series device reaches only through `STARTUPE2` (Section 8.2).

- **The RTL, restructured and debugged.** Eleven modules, one per file, with the memory map defined once and every copy of it checked. The bugs found and fixed include the arithmetic shift that shifted logically (`notes.md`, "SRL versus SRA: both shift arms are signed") and a simulation hang in the flash controller (`notes.md`, "Explicit initial values on div and sclk (simulation hang)"), and the design notes record the reason behind every subtle choice, such as the reset state (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").
- **The verification.** The golden trace and the regression, the ALU unit bench, ten lint rules, and the hooks that hold every commit to its gates (Chapter 9).
- **The board work.** The build flow from C to the board in both boot modes, every demo's output recorded from the board and checked against the PC, and a small design of its own that read and repaired the flash's Quad Enable bit (`notes.md`, "Writing the flash clears the Quad Enable bit").
- **The analysis.** The timing, utilisation and power of the routed design (Chapter 10), and this report, whose figures come from the real build and the real board.

1.3 What was built

The hardware is eleven Verilog modules, one per file, in one clock domain (Chapter 4). The core sees a single bus, a 32-bit address with a read strobe, write data with a byte mask and a busy input. `IOBus` decodes it into three regions by one address bit each: the RAM, the IO registers and the flash. The RAM is 6 KB of block RAM shared by instruction fetch and data. The IO registers are the LEDs, the UART's data register and its status word, and the flash controller turns a read in flash space into one SPI transaction and holds the core busy for its length.

The core (Chapter 6) executes RV32I and nothing beyond it: no multiplier, no interrupts, no control and status registers. An instruction takes four cycles, a store five and a load six, each cycle a memory or register-file read answered one cycle after it is asked. The same hardware boots two ways (Chapter 8): in BRAM boot the firmware image is read into the block RAM when the bitstream is synthesised. In flash boot the reset vector points into the flash and the core fetches each instruction from there in place.

The firmware is six demos in C and assembly, built by the RISC-V GCC with a start-up file and a linker script per boot mode. The repository (Chapter 3) holds the RTL, the firmware, the simulation, the Vivado scripts, this report and its design notes, with make targets that build and check the design. Verification (Chapter 9) is an ALU unit bench, a golden trace that the regression must reproduce byte for byte and cycle for cycle, and a lint. Each is a gate whose log a commit cites.

1.4 Results in brief

- **Size and speed.** The routed design is 984 LUTs, 244 flip-flops and two block RAM tiles, under half a percent of the XC7K325T. The 200 MHz clock is met with 0.219 ns of slack on the worst path, the load path (Chapter 10).
- **The board, the simulation and the source agree.** The first mandelbrot frame the board sent is byte for byte the golden trace's UART output, which is byte for byte what the C program prints when compiled natively. The board's frame time matches the simulated cycle count at 200 MHz (Section 1.1).

- **Flash boot works**, with one press of the reset button after each load (`notes.md`, "Flash boot loses its first fetch").
- **The demos are bound by the UART.** At 115 200 baud a byte takes 87 µs, and the mandelbrot frame spends about 956 of its 1031 ms sending. A faster core would not show on these demos.

1.5 How to read this report

Chapters 2 to 8 describe the system from the outside in: the board, the repository, the architecture, the RV32I instruction set, the core, the memory and IO, and the boot modes. Chapters 9 and 10 are the evidence beside Section 1.1: how the design is checked and what Vivado made of it. Chapter 11 lists ideas for extending the design, and Chapter 12 concludes.

The design notes, `docs/design/notes.md`, hold the full reasoning behind each module. The report states each choice and cites the notes' section by its heading, as (`notes.md`, "section").

2 The board

The Digilent Genesys 2 (Figures 1 and 2) carries a Kintex-7 XC7K325T, speed grade −2, in the 900-ball FFG900 package, with 1 GB of DDR3, HDMI and DisplayPort, Ethernet, audio, an FMC connector and five Pmods [3]. The design uses seven of its parts (Table 2.1). Five are wired to the design itself: the oscillator, the reset button, the LEDs, the USB-UART bridge and the flash connect to the nine ports of `SOC`, 16 FPGA pins in all. The other two, the USB-JTAG bridge and the mode jumper, connect to no port: they serve only to load a bitstream into the FPGA. Everything else is room to grow (Chapter 11).

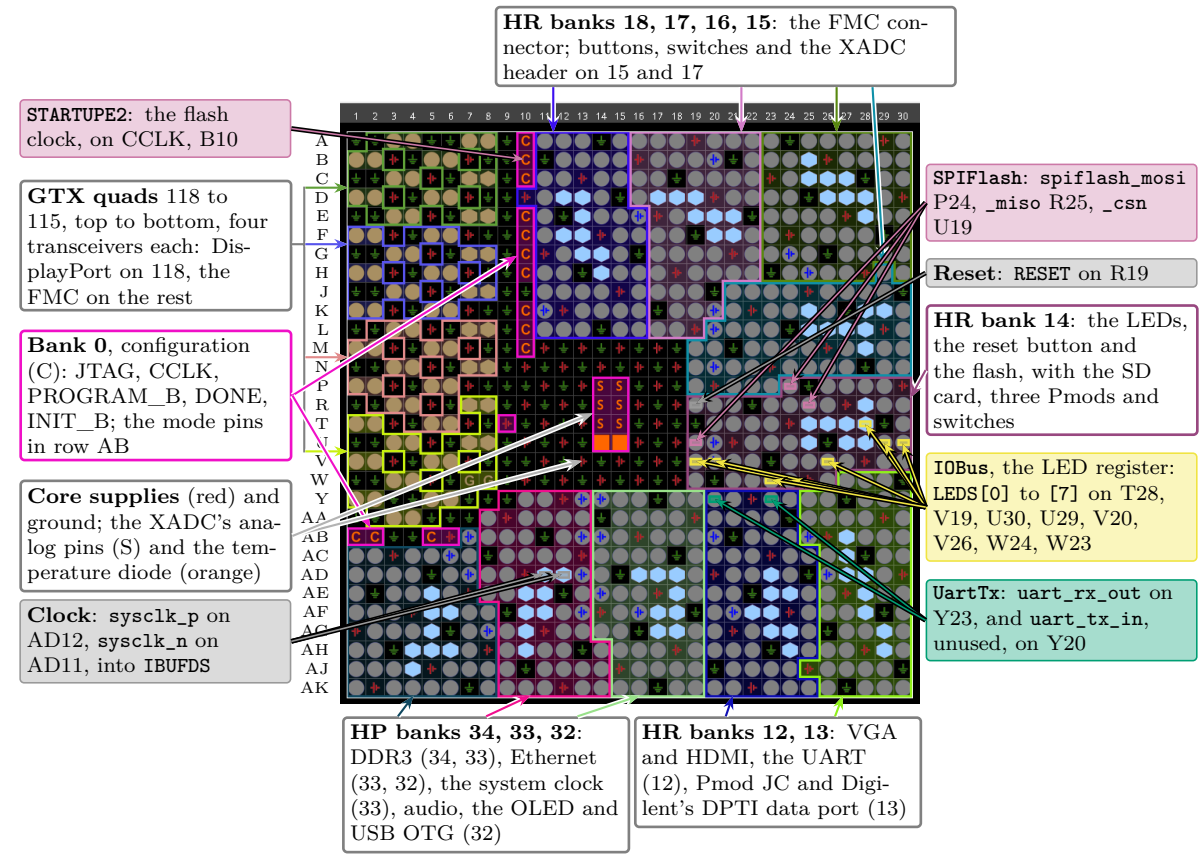
Part (schematic page)	FPGA board pin names	Design port
200 MHz LVDS oscillator, DSC1103 (IC23, p. 13)	AD12, AD11	`sysclk_p`, `sysclk_n`
CPU RESET button (BTN1, p. 11)	R19	`RESET`, low while pressed
LEDs LD0 to LD7 (p. 4)	T28, V19, U30, U29, V20, V26, W24, W23	`LEDS[0]` to `LEDS[7]`, high to light
USB-UART bridge, FT232RQ (IC16, p. 9), on the micro-USB port J15	Y23, Y20	`uart_rx_out`, `uart_tx_in`
QSPI flash, S25FL256S, 32 MB (IC26, p. 15)	U19, P24, R25, B10	`spiflash_csn`, `spiflash_mosi`, `spiflash_miso`, and the clock through `STARTUPE2`
USB-JTAG bridge, FT2232HQ (read from its marking: the schematic draws only its JTAG nets, p. 15), on the micro-USB port J17	F10, E10, H10, G10 (TMS, TCK, TDI, TDO)	none: it loads bitstreams
Mode jumper JP5 (p. 15)	AB1, AB2, AB5 (M2, M1, M0)	none: it picks what configures the FPGA at power-on

Table 2.1: The Genesys 2 parts the design uses. The pin names are the FPGA package's ball names, as the FPGA symbol of Digilent's schematic, revision H.1 [4], labels them, in the order of the ports beside them. Parts and pages are the schematic's, and the pins of the design's ports are also those of `constraints/Genesys-2-Master.xdc`.

The package (Figure 2.1). The FPGA's silicon die sits inside a *package*, the square black part on the board, which connects it to the board's copper. The FFG900 is a *ball grid array*: instead of legs along its edges it has 900 small solder balls on its underside, a grid of 30 by 30 at 1 mm spacing, each soldered to a pad on the board. A ball is named like a square of a chessboard, by its row letter and its column number: AD12 is row AD, column 12. The rows run from A to AK, with I, O, Q, S, X and Z left out so that no letter can be read as a digit. Not every ball is a signal. Many are supply and ground, which share the chip's current between them and keep its supply steady, and others serve configuration, the transceivers or the on-chip analog-to-digital converter. Of the 900, 500 are *user I/O*, pins a design may use as it likes.

User I/O come in *banks*. A bank is 50 pins that share one output supply, VCCO, so all the pins of a bank work at one voltage: up to 3.3 V in the high-range (HR) banks and up to 1.8 V in the high-performance (HP) banks [9]. The FFG900 has 15 banks in all. Ten hold the 500 user I/O: the seven HR banks 12 to 18 and the three HP banks 32 to 34. Bank 0 holds the

dedicated configuration pins. The other four, 115 to 118, are GTX quads: a GTX quad is four multi-gigabit transceivers, each with a transmit and a receive pair. The supplies of the FPGA's core, in the middle of the package, belong to no bank.



User I/O pin: a pin free for a design to use.

Clock-capable I/O pin: a user I/O that can also drive the FPGA's clock network directly, so a clock from outside must arrive on one.

VREF-capable I/O pin: a user I/O that can instead carry a reference voltage for its bank, the threshold that inputs of standards such as DDR3's are compared against. When it does not, it is an ordinary user I/O.

A design pin, in its module's colour: a user I/O that carries one of the design's ports. Vivado draws a bar over the ball in the colour of the module that drives or reads the port: yellow for the LED register of `IOBus`, green for `UartTx`, pink for `SPIFlash`, grey for the clock and reset.

Supply pin: carries the chip's current in.

Ground: carries the chip's current out.

Configuration pin: a dedicated pin of bank 0, for loading a bitstream.

Transceiver pin: a signal pin of a GTX quad.

Transceiver calibration pin: the two pins across which the board fits a precision resistor, which the transceivers match their termination to.

XADC analog pin: the inputs and supply of the on-chip analog-to-digital converter.

Temperature diode: the two ends of a diode on the die, whose voltage tells the die's temperature.

Figure 2.1: Vivado's Package view of the FFG900 package, from the routed design: the 900 balls seen from above, column numbers 1 to 30 across the top and row letters A to AK down the left, each ball drawn as a symbol of the key under the picture. Each coloured outline groups the balls of one bank, and the labels around the picture name the banks and say what each serves on the Genesys 2 [4]. The design's 16 ports use 16 of the 500 user I/O, about 3%, drawn in their modules' colours, and each module's box points at its pins. They sit in three banks: the LEDs, the reset button and the flash on bank 14, the UART on bank 12 and the clock on bank 33. The flash's clock has no user pin: it leaves on CCLK, a configuration pin of bank 0, through `STARTUPE2`. `uart_tx_in`, on Y20, has no load and so no IOB, but keeps its pin.

Vivado's numbers depend on its version: another release may place and route the same RTL differently, so the numbers here and in Chapter 10 hold for Vivado 2025.2.

3 Repository tour

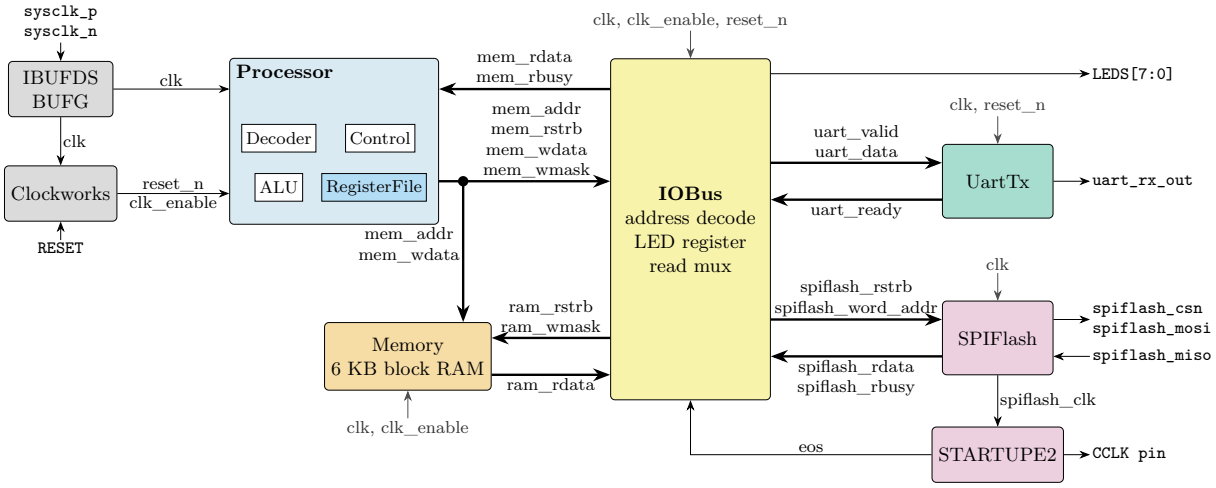
The repository is a plain source tree. The Vivado project is not committed: a script generates it into `build/vivado/`, referring to the sources where they are, and git ignores what the tools generate. Table 3.1 gives each folder at a glance. Each tracked folder has a README of its own, the place to go next: it lists the folder's files, says where to start reading and which make targets serve the folder.

Folder	What it holds
`rtl/`	The synthesisable Verilog: the SoC's top level, the core and the peripherals, one module per file, with the memory map and the firmware image
`fw/`	The demo programs in C and assembly, and what builds them: start-up code, linker scripts, a small runtime and the image converters
`sim/`	The simulation: the SoC testbench, its golden trace and the regression script, and the ALU unit bench
`constraints/`	The board's pins and the 200 MHz clock, for Vivado
`vivado/`	The Tcl scripts that generate the Vivado project and build the bitstream
`docs/`	The reference numbers, this report and its design notes, and the demo animations that the README plays
`scripts/`	The lint and the git hooks that hold every commit to its gates
`build/`	Everything generated: the Vivado project, `SOC.bit`, the gates' logs. Not tracked

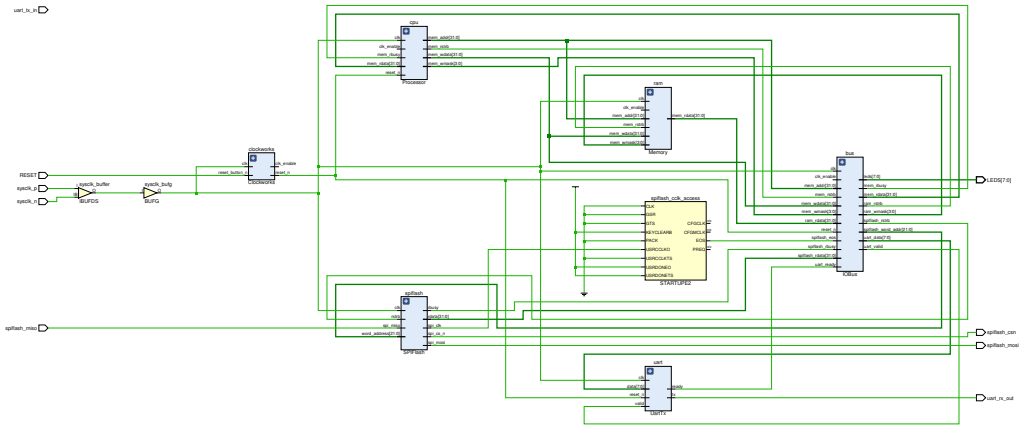
Table 3.1: The folders at a glance; each tracked folder's README says more.

4 System architecture

The SoC is six modules in one clock domain, wired together by `SOC`, the top level, with four more inside the core. The core sees one bus, and everything else hangs off that bus. Figure 4.1 is the block diagram, drawn and as Vivado inferred it, and the sections below give the bus, the memory map and the clock and reset.



(a) The block diagram, from `rtl/SOC.v`, each wire under its name there. The thick arrows are the core's bus and the handshakes `IOBus` derives from it, one arrow for each direction. The core's address and write data are shared wires: at the dot they branch to the RAM as well as to `IOBus`. `IOBus` decodes the address and strobes only the block it selects: the RAM's read strobe and byte mask, the UART's send pulse with the low byte of the write data, or the flash controller's read strobe with the word address. It holds the LED register and multiplexes the read data back to the core.



(b) Vivado's RTL schematic of the same top level, elaborated with `synth_design -rtl`: the six instances under their names in `SOC.v`, and the three primitives, `IBUFDS` and `BUFG` on the clock and `STARTUPE2` on the flash clock.

Figure 4.1: The SoC as drawn and as Vivado inferred it.

4.1 The bus and the modules

The core, `Processor`, sees one bus and nothing else: a 32-bit byte address, a one-cycle read strobe, 32 bits of read data, 32 bits of write data with a four-bit byte mask, and a busy input.

"One bus" means one set of ports whose far side the core does not know. The address and the write data are plain wires that reach every block that needs them, `IOBus` and the RAM alike, as the dot in Figure 4.1a shows. A block acts on them only when its strobe or its mask says so, and `IOBus` is what raises those for the region the address falls in, and what picks the block whose read data returns to the core. There is no acknowledge. A read requested with `mem_rstrb` is answered on `mem_rdata` in the next cycle unless `mem_rbusy` is high, and a write lands on the edge where `mem_wmask` is non-zero. Every address falls in some region, so no access is refused (`notes.md`, "Processor bus has no notion of RAM versus device").

Everything around the core follows from that contract. `IOBus` decodes the address into three regions, forwards the strobe and the mask to the RAM, holds the LED register, turns a store into the UART's send pulse and a flash read into an SPI transaction, and multiplexes the read data back. `Memory` is 6 KB of block RAM with one port, shared by instruction fetch and by loads and stores. Its read is registered, which is why the core waits a cycle after every read, and its write lands on the same edge, which is why a store needs no wait (`notes.md`, "Read and write timing are not symmetric"). `UartTx` sends 8N1 at 115 200 baud and `SPIFlash` reads one 32-bit word per transaction, at a sixth of the system clock. The flash controller is the only block that stalls the core, by holding `mem_rbusy` for the length of a transaction (`notes.md`, "Only flash reads stall the core, read mux"). `Clockworks` is among the instances of Figure 4.1b but contributes no cell to the netlist, as Section 4.3 explains.

4.2 Memory map and address decode

Table 4.1 is the memory map, defined once in `rtl/defines.vh`. The firmware carries copies of its values in `fw/io.h` and the start-up code, and a lint rule holds every copy equal to the defines (Chapter 9).

Byte address	Selected by	Region or register	Access
`0x0000_0000`–`0x0000_17FF`	no region bit	RAM, 6 KB (1536 words)	read, write, fetch
`0x0040_0000`	bit 22	`IO_BASE`, the IO registers	
`+ 0x04`	bit 2	`IO_LEDS`, the LED register	write
`+ 0x08`	bit 3	`IO_UART_DAT`, send one byte	write
`+ 0x10`	bit 4	`IO_UART_CNTL`, status, bit 9 = busy	read
`0x0100_0000`–`0x01FF_FFFF`	bit 24	SPI flash, a 16 MB window on the chip	read, fetch
`0x01D0_0000`–`0x01FF_FFFF`		the code window, 3 MB, where flash boot starts	

Table 4.1: The memory map (`rtl/defines.vh`). The reset vector is `0x0000_0000` in BRAM boot and `0x01D0_0000` in flash boot (Chapter 8).

A region is chosen by one address bit, not by a comparator: bit 24 selects the flash, bit 22 the IO registers when bit 24 is clear, and an address with neither set is RAM. Inside the IO region each register owns one bit as well, so a store to `IO_BASE + 4` lights the LEDs and a store to `IO_BASE + 8` sends a byte. The whole decode is a few bit tests in `IOBus` (`notes.md`, "Address decode: one bit per region, one-hot IO registers, why bit 24").

Three consequences matter when writing firmware. Nothing is rejected: an address past the RAM's 6 KB still selects the RAM, and what such an access does is undefined. Only the UART status register answers a read, and every other IO address reads as zero. And a write to flash space does nothing. To add a peripheral, give it the next free bit of the IO word address, bit 5 at byte offset `0x20`, decode it in `IOBus` as the three existing ones are decoded, add it to the read mux if it answers reads, and add its copy in `fw/io.h` to the lint rule.

4.3 Clock and reset

The board's 200 MHz oscillator drives a differential pair, so the clock enters through `IBUFDS` and `BUFG` (`notes.md`, "Genesys 2 differential clock input"). That net, `clk`, is the only clock in the design, and its 5 ns period is the one timing constraint (Chapter 10).

`Clockworks` derives two signals from the clock and the button. `clk_enable` is a gearbox: with the parameter `SLOW` set to n, it is high for one cycle in 2^n, and the core, the bus and the RAM advance only on those cycles, while the UART and the flash controller run every cycle, so the baud rate and the SPI clock stay right (`notes.md`, "Gearbox counter"). The shipped design sets it to 0, `clk_enable` is the constant 1, and the module synthesises to nothing.

The reset input is asynchronous and the resets it drives are synchronous (`notes.md`, "Synchronous reset and the clock enable"). A press or a release that lands close to an edge can reach one register that resets a cycle before another, or leave one metastable for a moment. A two-flip-flop synchroniser in `Clockworks` would remove that risk, and the design has none. The core's registers also declare their reset value as their initial value, which is what configuration loads into a 7-series flip-flop, so the core leaves configuration in its reset state and starts without a press of the button (`notes.md`, "Power-on state matches the reset state"). In flash boot a press is still needed on the board, for a reason in the flash controller's start-up (`notes.md`, "Flash boot loses its first fetch"). The FSM resets into `WAIT_DATA`, which asserts no strobe and falls through to `FETCH_INSTR`, not into `FETCH_INSTR` itself (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").

5 The RV32I instruction set

RV32I is the base integer instruction set of RISC-V [2], the one this core executes and the one the firmware is compiled for, with `-march=rv32i -mabi=ilp32`. It has 32 registers of 32 bits and 40 instructions, each one 32 bits long. It is a load-store architecture: only the loads and the stores touch memory, and every other instruction works on registers and constants. This chapter gives what the later chapters rely on: the registers, how an instruction is written, how its 32 bits are laid out, what each instruction does in this core, and how a function call uses them.

5.1 Registers and assembly syntax

The registers are `x0` to `x31`. `x0` always reads as zero, and a write to it is discarded, which makes it useful as a source of zero and as a destination for a result nobody needs. The program counter, `pc`, is separate and is not one of the 32. The calling convention (Section 5.4) gives each register a role and an ABI name, the name the compiler and the disassembler print (Table 5.1).

Registers	ABI names	Role
`x0`	`zero`	always zero
`x1`	`ra`	return address, written by a call
`x2`	`sp`	stack pointer, the top of the RAM at start-up
`x3`	`gp`	global pointer, set to `IO_BASE` by this design's start-up code
`x4`	`tp`	thread pointer, unused here
`x5`–`x7`, `x28`–`x31`	`t0`–`t2`, `t3`–`t6`	temporaries, which a call may overwrite
`x8`, `x9`, `x18`–`x27`	`s0`–`s1`, `s2`–`s11`	saved registers, which a call preserves (`s0` also `fp`)
`x10`–`x17`	`a0`–`a7`	arguments, `a0` and `a1` also the return value

Table 5.1: The 32 registers and their roles in the calling convention.

5.2 Instruction formats

Every instruction is one of six formats, R, I, S, B, U and J, which differ in where they keep their constant, the *immediate* (Figure 5.1). The fields are:

- **opcode**, bits 6 to 0, the instruction's class: register arithmetic, arithmetic with an immediate, load, store, branch and so on. Its two low bits are `11` in every 32-bit instruction.
- **rd**, **rs1** and **rs2**, five bits each, the numbers of the destination and the two source registers. They sit at the same bit positions in every format that has them, so the register file can be read before the instruction is fully decoded. This core reads `rs1` and `rs2` in `FETCH_REGS` for every instruction, whether it uses them or not (Chapter 6).
- **funct3** and **funct7**, which choose the operation within a class: `funct3` picks one of the eight ALU operations, the branch condition or the load and store width, and bit 30, in `funct7`, tells `SUB` from `ADD` and `SRA` from `SRL`.
- **imm**, the immediate, a signed constant. Its sign is always bit 31, so it can be sign-extended before the format is known. I and S immediates are 12 bits, from -2048 to 2047. B and J immediates are byte offsets with bit 0 left out, since every instruction starts at an even address: a branch reaches ± 4 KB and `jal` ± 1 MB. A U immediate is the upper 20 bits of a 32-bit value.

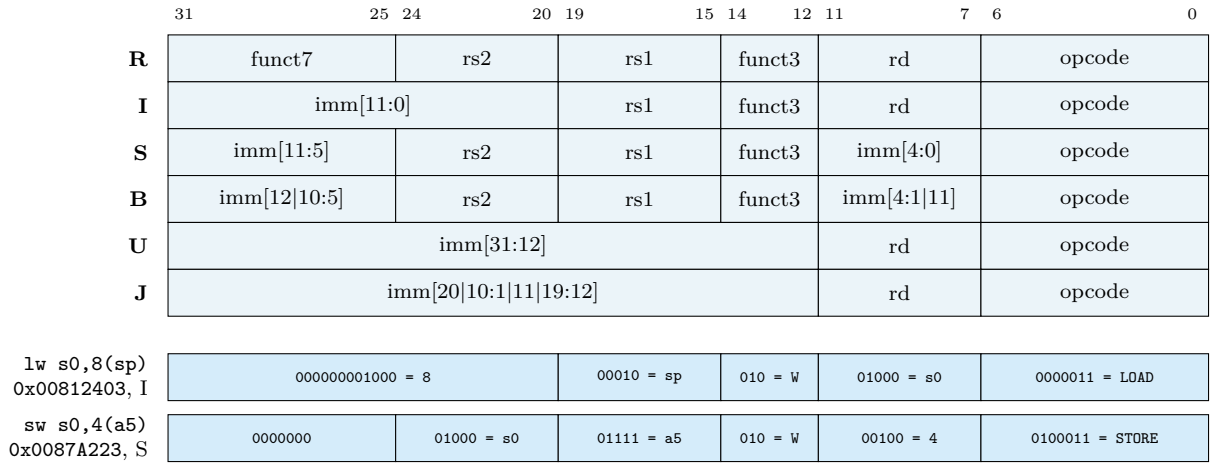


Figure 5.1: The six instruction formats, bit 31 on the left, and two instructions of the mandelbrot demo decoded: the load and the store that writes the LEDs, both of Figure 6.3. `imm[12|10:5]` means immediate bit 12, then bits 10 to 5. B and J scatter their immediate's bits so that each immediate bit sits at the same instruction bit in as many formats as possible.

5.3 The instructions

Table 5.2 lists the 40 instructions, one a row, with what each one does and how many cycles it takes in this core when the instruction is fetched from the RAM (Table 6.1). In the operations, `imm` is the sign-extended immediate, `mem` the memory, and `x0` the register that reads as zero. RV32I has no multiply and no divide: the compiler turns every `*`, `/` and `%` in C into a call into `libgcc`, which does it with shifts and adds. After the instructions, the table lists the pseudo-instructions and aliases that the GNU assembler (binutils 2.42, `-march=rv32i`) accepts, each with the real instructions it writes in its place, as `objdump -M no-aliases` shows them, and so the cycles they cost.

Table 5.2: The RV32I instructions and what this core does with each, then the pseudo-instructions and aliases of the GNU assembler. Fmt is the format of Figure 5.1, and the cycles are from the RAM: every word fetched or loaded from the flash adds an SPI transaction (Chapter 7).

Instruction	Fmt	Operation	Cycles
Register and register			
`ADD rd,rs1,rs2`	R	`rd` ← `rs1` + `rs2`	4
`SUB rd,rs1,rs2`	R	`rd` ← `rs1` − `rs2`	4
`AND rd,rs1,rs2`	R	`rd` ← `rs1` AND `rs2`, bit by bit	4
`OR rd,rs1,rs2`	R	`rd` ← `rs1` OR `rs2`, bit by bit	4
`XOR rd,rs1,rs2`	R	`rd` ← `rs1` XOR `rs2`, bit by bit	4
`SLL rd,rs1,rs2`	R	`rs1` shifted left by `rs2[4:0]` places, zeros in	4
`SRL rd,rs1,rs2`	R	`rs1` shifted right by `rs2[4:0]` places, zeros in	4
`SRA rd,rs1,rs2`	R	`rs1` shifted right by `rs2[4:0]` places, copies of its sign bit in	4
`SLT rd,rs1,rs2`	R	`rd` ← 1 if `rs1` < `rs2` as signed numbers, else 0	4
`SLTU rd,rs1,rs2`	R	`rd` ← 1 if `rs1` < `rs2` as unsigned numbers, else 0	4
Register and immediate			
`ADDI rd,rs1,imm`	I	`rd` ← `rs1` + `imm`	4
`ANDI rd,rs1,imm`	I	`rd` ← `rs1` AND `imm`, bit by bit	4
`ORI rd,rs1,imm`	I	`rd` ← `rs1` OR `imm`, bit by bit	4
`XORI rd,rs1,imm`	I	`rd` ← `rs1` XOR `imm`, bit by bit	4
`SLLI rd,rs1,sh`	I	`rs1` shifted left by `sh`, 0 to 31 places, zeros in	4
`SRLI rd,rs1,sh`	I	`rs1` shifted right by `sh`, 0 to 31 places, zeros in	4

Instruction	Fmt	Operation	Cycles
`SRAI rd,rs1,sh`	I	`rs1` shifted right by `sh`, 0 to 31 places, copies of its sign bit in	4
`SLTI rd,rs1,imm`	I	`rd` ← 1 if `rs1` < `imm` as signed numbers, else 0	4
`SLTIU rd,rs1,imm`	I	`rd` ← 1 if `rs1` < `imm` as unsigned numbers, else 0	4
Upper immediate			
`LUI rd,imm`	U	`rd` ← `imm` ≪ 12	4
`AUIPC rd,imm`	U	`rd` ← `pc` + (`imm` ≪ 12)	4
Jumps			
`JAL rd,label`	J	`rd` ← `pc` + 4, `pc` ← `pc` + `imm`: a call when `rd` is `ra`	4
`JALR rd,imm(rs1)`	I	`rd` ← `pc` + 4, `pc` ← `rs1` + `imm` with bit 0 cleared: a return, or a call through a pointer	4
Branches			
`BEQ rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` = `rs2`	4
`BNE rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≠ `rs2`	4
`BLT rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` < `rs2` as signed numbers	4
`BGE rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≥ `rs2` as signed numbers	4
`BLTU rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` < `rs2` as unsigned numbers	4
`BGEU rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≥ `rs2` as unsigned numbers	4
Loads			
`LW rd,imm(rs1)`	I	`rd` ← the word at `rs1` + `imm`	6
`LH rd,imm(rs1)`	I	`rd` ← the halfword at `rs1` + `imm`, sign-extended	6
`LHU rd,imm(rs1)`	I	`rd` ← the halfword at `rs1` + `imm`, zero-extended	6
`LB rd,imm(rs1)`	I	`rd` ← the byte at `rs1` + `imm`, sign-extended	6
`LBU rd,imm(rs1)`	I	`rd` ← the byte at `rs1` + `imm`, zero-extended	6
Stores			
`SW rs2,imm(rs1)`	S	the word at `rs1` + `imm` ← `rs2`	5
`SH rs2,imm(rs1)`	S	the halfword at `rs1` + `imm` ← the low halfword of `rs2`, the rest of the word left as it was	5
`SB rs2,imm(rs1)`	S	the byte at `rs1` + `imm` ← the low byte of `rs2`, the rest of the word left as it was	5
Ordering and system			
`FENCE`	I	orders memory accesses between cores and devices. Here a no-op	4
`ECALL`	I	calls the operating system. Here it stops the core	4, repeated
`EBREAK`	I	calls the debugger. Here it stops the core	4, repeated
Pseudo-instructions and aliases: what the assembler writes for each			
`nop`	–	`ADDI x0,x0,0`	4
`li rd,imm`	–	`ADDI rd,x0,imm` when `imm` fits in 12 bits, −2048 to 2047. Otherwise `LUI rd,hi` then `ADDI rd,rd,lo`: `lo` is the low 12 bits of `imm` read as a signed number, `hi` is (`imm` − `lo`) ≫ 12, and the `ADDI` is left out when `lo` is 0	4 or 8
`la rd,symbol`	–	`AUIPC rd,hi` then `ADDI rd,rd,lo`: `hi` and `lo` split the distance from the `AUIPC` to the symbol as `li` splits `imm`. The symbol's address, without position-independent code, as the firmware is built	8
`lla rd,symbol`	–	`AUIPC rd,hi` then `ADDI rd,rd,lo`, as `la`	8
`mv rd,rs`	–	`ADDI rd,rs,0`	4
`not rd,rs`	–	`XORI rd,rs,-1`	4
`neg rd,rs`	–	`SUB rd,x0,rs`	4
`seqz rd,rs`	–	`SLTIU rd,rs,1`: 1 if `rs` is zero	4
`snez rd,rs`	–	`SLTU rd,x0,rs`: 1 if `rs` is not zero	4
`sltz rd,rs`	–	`SLT rd,rs,x0`: 1 if `rs` is negative	4
`sgtz rd,rs`	–	`SLT rd,x0,rs`: 1 if `rs` is positive	4
`zext.b rd,rs`	–	`ANDI rd,rs,255`	4
`sext.b rd,rs`	–	`SLLI rd,rs,24` then `SRAI rd,rd,24`	8
`sext.h rd,rs`	–	`SLLI rd,rs,16` then `SRAI rd,rd,16`	8

Instruction	Fmt	Operation	Cycles
`zext.h rd,rs`	–	`SLLI rd,rs,16` then `SRLI rd,rd,16`	8
`beqz rs,label`	–	`BEQ rs,x0,label`	4
`bnez rs,label`	–	`BNE rs,x0,label`	4
`blez rs,label`	–	`BGE x0,rs,label`	4
`bgez rs,label`	–	`BGE rs,x0,label`	4
`bltz rs,label`	–	`BLT rs,x0,label`	4
`bgtz rs,label`	–	`BLT x0,rs,label`	4
`bgt rs,rt,label`	–	`BLT rt,rs,label`	4
`ble rs,rt,label`	–	`BGE rt,rs,label`	4
`bgtu rs,rt,label`	–	`BLTU rt,rs,label`	4
`bleu rs,rt,label`	–	`BGEU rt,rs,label`	4
`j label`	–	`JAL x0,label`	4
`jal label`	–	`JAL ra,label`	4
`jr rs`	–	`JALR x0,0(rs)`	4
`jalr rs`	–	`JALR ra,0(rs)`	4
`ret`	–	`JALR x0,0(ra)`	4
`call label`	–	`AUIPC ra,hi` then `JALR ra,lo(ra)`, `hi` and `lo` as in `la`	8
`tail label`	–	`AUIPC t1,hi` then `JALR x0,lo(t1)`: a jump that does not return	8
`fence`	–	`FENCE iorw,iorw`: every kind of access ordered	4
`lw rd,symbol`	–	`AUIPC rd,hi` then `LW rd,lo(rd)`	10
`lh rd,symbol`	–	`AUIPC rd,hi` then `LH rd,lo(rd)`	10
`lhu rd,symbol`	–	`AUIPC rd,hi` then `LHU rd,lo(rd)`	10
`lb rd,symbol`	–	`AUIPC rd,hi` then `LB rd,lo(rd)`	10
`lbu rd,symbol`	–	`AUIPC rd,hi` then `LBU rd,lo(rd)`	10
`sw rs,symbol,rt`	–	`AUIPC rt,hi` then `SW rs,lo(rt)`	9
`sh rs,symbol,rt`	–	`AUIPC rt,hi` then `SH rs,lo(rt)`	9
`sb rs,symbol,rt`	–	`AUIPC rt,hi` then `SB rs,lo(rt)`	9

Pseudo-instructions. Pseudo-instructions are made by the GNU assembler, which builds the firmware: it writes the real instructions of each one's row in its place, so they exist only in assembly text and the core never sees one. The project's other assembler, the tiny Verilog-macro assembler of `sim/asm/` that comes from Bruno Levy's series and writes small test programs, cannot make most of them: of the table's pseudo-instructions it knows only `nop`, `li`, `mv`, `j`, `jr`, `ret`, `call`, `beqz`, `bnez` and `bgt`.

FENCE. Its opcode sets none of `Decoder`'s class flags, so the core treats it as an ALU instruction whose `rd` is `x0`: a no-op, which is all a `FENCE` has to do with one core.

ECALL and EBREAK. `EXECUTE` leaves `pc` unchanged for the SYSTEM opcode, so the core fetches the instruction again, forever: a halt that the reset button ends. The start-up code ends with `EBREAK`, should `main` return.

Left out. No extension: no M, A, F, D or C, and no Zicsr. The CSR instructions share the SYSTEM opcode and stop the core as `ECALL` does. There are no privilege modes, no exceptions and no interrupts: an illegal instruction is executed as `FENCE` is, and a misaligned load or store is not trapped, so its result is wrong, though code built from C never makes one for aligned data. The illegal instruction needs no mechanism of its own: an unknown opcode raises none of `Decoder`'s class flags, so every choice in `Control` and `Processor` falls to its default, the one an ALU instruction takes. The FSM goes from `EXECUTE` back to `FETCH_INSTR`, `pc` advances by 4, and `rd` takes the ALU's result on `rs1` and the I-immediate. For `FENCE` `rd` is `x0`, so nothing changes; an illegal instruction whose `rd` field is not zero overwrites that register.

5.4 The calling convention

RV32I has no call, return, push or pop instruction: a call is a `JAL` or `JALR` that writes the return address to `ra`, a return is a `JALR` through `ra`, and the stack is `sp` and the loads and stores. Everything else about a call is the calling convention of the RISC-V ABI for `ilp32`, which the compiler follows and the hand-written assembly in `fw/` follows too, so that C and assembly call each other. It divides the work between the two sides of a call:

- **The caller** puts the first eight arguments in `a0` to `a7`, any more on the stack, and calls with `call`, `AUIPC` then `JALR`, which leaves the return address in `ra`. After the call the result is in `a0`, and `a1` too for a 64-bit one. The caller must assume that the call has overwritten `ra`, `t0` to `t6` and `a0` to `a7`, the *caller-saved* registers, so a value it needs after the call it keeps in an `s` register or in its own stack frame.
- **The callee** may overwrite the caller-saved registers, but must return with `sp` and `s0` to `s11`, the *callee-saved* registers, as it found them. A function that uses an `s` register saves it first, and so does one that calls another function, which overwrites `ra`. Its *prologue* takes a stack frame, by subtracting from `sp`, and stores those registers in it; its *epilogue* loads them back, gives the frame back and returns with `ret`.

The stack grows down from the top of the RAM, where the start-up code points `sp`, and `sp` stays a multiple of 16 bytes, so every frame is too. Table 5.3 below serves as an example:

Address	Instruction	What it does
Prologue		
42c	`addi sp,sp,-16`	takes a 16-byte frame
430	`sw s0,8(sp)`	saves the caller's `s0`, which the body uses
434	`sw ra,12(sp)`	saves the return address, which the calls overwrite
Body		
438	`mv s0,a0`	keeps the pointer, the first argument, where `putchar` preserves it
43c	`lbu a0,0(a0)`	the first character, as `putchar`'s argument
440	`beqz a0,458`	an empty string: straight to the epilogue
444	`auipc ra,0x0`	`call putchar`, first half: `ra` = `pc` = `0x444`
448	`jalr -68(ra)`	second half: to `0x444` − 68 = `0x400`, `putchar`, with `ra` = `0x44c`
44c	`lbu a0,1(s0)`	the next character
450	`addi s0,s0,1`	
454	`bnez a0,444`	and again, until the terminating zero
Epilogue		
458	`lw ra,12(sp)`	restores the return address
45c	`lw s0,8(sp)`	restores the caller's `s0`
460	`addi sp,sp,16`	gives the frame back
464	`ret`	`JALR x0,0(ra)`: back to the caller

Table 5.3: `print_string` of `fw/print.c` in the mandelbrot demo, from `objdump -d mandelbrot.bram.elf`: `for(const char* p = s; *p; ++p) putchar(*p);`, compiled with `-O2`.

6 The multi-cycle core

`Processor` executes RV32I and nothing beyond it: no M extension, no interrupts, no control and status registers, and `ECALL` and `EBREAK` stop the core (Chapter 5). It is a multi-cycle core: one instruction at a time, several clock cycles each, one FSM state per cycle. `Control` is the FSM, `Decoder`, `ALU` and `RegisterFile` are what their names say, and `Processor` holds `pc`, the instruction register and the glue between the other four.

6.1 The state machine

An instruction takes several cycles because the RAM answers a read one cycle after it is requested, and the register file's read is synchronous too, so each needs a state of its own (`notes.md`, "Why one instruction takes several cycles"). A store needs no state after it, as the RAM writes on the edge that ends `STORE` (`notes.md`, "STORE needs no wait state"). Figure 6.1 shows the seven states and everything each one does.

`Control` decodes the state into four one-cycle strobes, `latch_instr`, `read_regs`, `update_pc` and `write_rd`, already qualified with `clk_enable` and `reset_n`. Table 6.1 gives the cost per instruction class.

Instruction class	States	Cycles
ALU with a register or an immediate, LUI, AUIPC, JAL, JALR, branch	FI WI FR EX	4
Load: LB, LH, LW, LBU, LHU	FI WI FR EX LD WD	6
Store: SB, SH, SW	FI WI FR EX ST	5
SYSTEM: ECALL, EBREAK	FI WI FR EX, `pc` unchanged	4, repeated

Table 6.1: Cycles per instruction class from the RAM. Every word fetched or loaded from the flash adds the SPI transaction to its wait state (Chapter 7).

Two of the four cycles of the commonest class are the fetch. A pipelined core would overlap the fetch of one instruction with the execution of the last and retire close to one a cycle, a gain of four to six on code bound by computation. The demos of Section 1.1 are bound by the UART, so for them the multi-cycle core costs nothing visible. Pipelining is the first extension in Chapter 11.

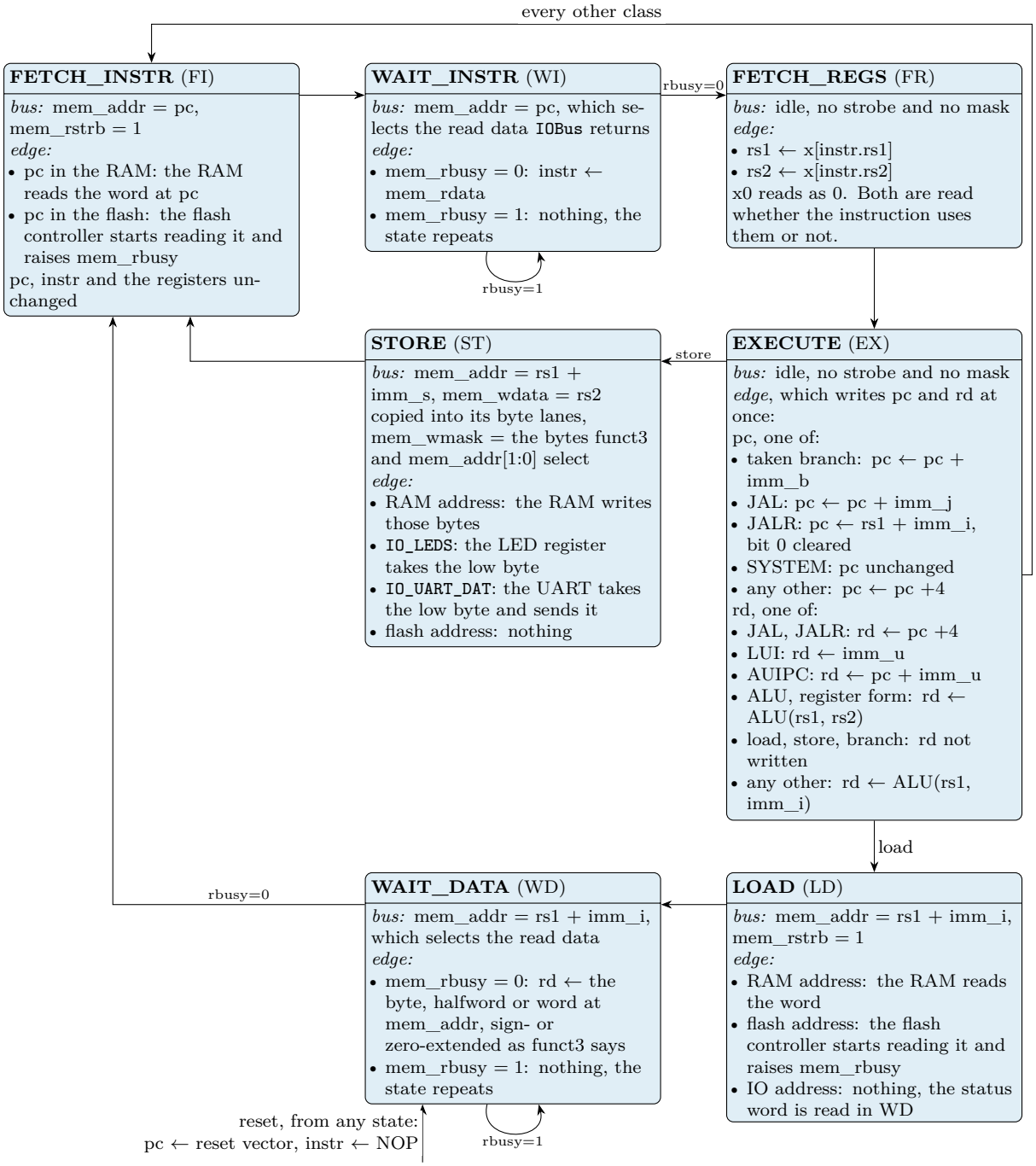


Figure 6.1: The control FSM of `rtl/Control.v` and what each state does to the datapath of `rtl/Processor.v`: *bus* is what the core drives on the bus in that state, *edge* what the clock edge that ends the state writes. Every state lasts one cycle but the two waits, which hold while the flash controller is busy; "rbusy" on the arrows is mem_rbusy. The two-letter codes are the ones Figure 6.3 uses. Reset puts the FSM in `WAIT_DATA`, which asserts no strobe, so a core held in reset leaves the bus idle. Its write to rd goes to `x0`, the destination of the NOP that instr resets to, and when reset ends it falls through to `FETCH_INSTR` at the reset vector (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").

6.2 The datapath

`Decoder` is purely combinational on the instruction register: the ten class flags from the opcode, disjoint, so exactly one is high for every instruction the core implements, the register indices

and the `funct3` and `funct7` fields, and the five immediates of Figure 5.1, sign-extended from bit 31. It also settles the two meanings of `funct7[5]` (`notes.md`, "ADD versus SUB: funct7[5] alone is not enough").

`RegisterFile` is the 32 integer registers in LUT RAM: a synchronous read of `rs1` and `rs2` on `read_regs`, a synchronous write of `rd` on `write_rd`, and `x0` never written and read as zero explicitly (`notes.md`, "x0 reads as zero explicitly"). `ALU` is combinational (Figure 6.2): its second operand is `rs2` for register instructions and the I-immediate for everything else, `funct3` picks one of the eight operations, and a second case on `funct3` computes the six branch conditions, with a default of zero so that no latch is inferred (`notes.md`, "Branch comparator default arm"). Both arms of the right shift are signed, or `SRA` shifts logically (`notes.md`, "SRL versus SRA: both shift arms are signed").

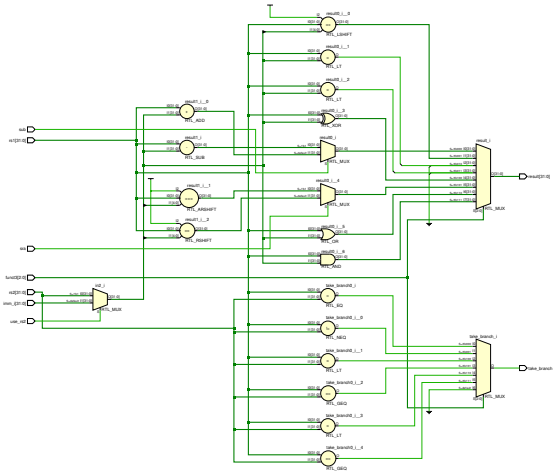


Figure 6.2: Vivado's RTL schematic of `rtl/ALU.v`, elaborated as its own top. Upper half: the operand multiplexer on the left, then the adder and subtractor, the shifters, the comparators and the logic operations, into the result multiplexer on the right. Lower half: the branch comparators, into the multiplexer that yields `take_branch`. After synthesis nearly all of this merges into the LUTs around the register file (Chapter 10).

The rest of `Processor` is the glue between these. A multiplexer chooses the next `pc`: the branch target when a branch is taken, the jump target for `JAL`, `rs1` plus the immediate with bit 0 cleared for `JALR`, otherwise `pc + 4`. An adder forms the load or store address from `rs1` and the I- or S-immediate. The memory moves whole words, so a byte or halfword load picks its lane by the address's low two bits and sign-extends unless `funct3[2]` says not to, and a store broadcasts `rs2` into every lane it could land in and lets the byte mask select (`notes.md`, "Store data is broadcast to every lane, the mask selects"). The write-back multiplexer gives `rd` its value: `pc + 4` for the jumps, the U-immediate for `LUI`, `pc` plus it for `AUIPC`, the aligned load data, or the ALU result. The design's worst timing path runs through this glue, from the instruction register through the load-store address adder, the region decode and the load lanes to the register file's write port (`notes.md`, "The worst path ends in the register-file write").

6.3 One instruction through the states

Figure 6.3 is the simulated core at the end of `main`'s prologue in the mandelbrot demo, where the frame counter, just stored to the stack, is loaded back and written to the LEDs: four instructions, 19 cycles, every state of Figure 6.1.

The load takes cycles 0 to 5. In `FETCH_INSTR` the bus address is `pc`, `0x278`, with the strobe. In `WAIT_INSTR` the RAM answers with the instruction word, `0x00812403` (Figure 5.1), and `latch_instr` moves it into `instr`. From then on the bus address is the load-store address,

`sp + 8`. `EXECUTE` advances `pc` to `0x27c`, `LOAD` asserts the strobe at the data address, and `WAIT_DATA` sees the word, 0, and `write_rd` writes it to `s0`. Figure 6.3, below, shows the rest of the instructions the same way.

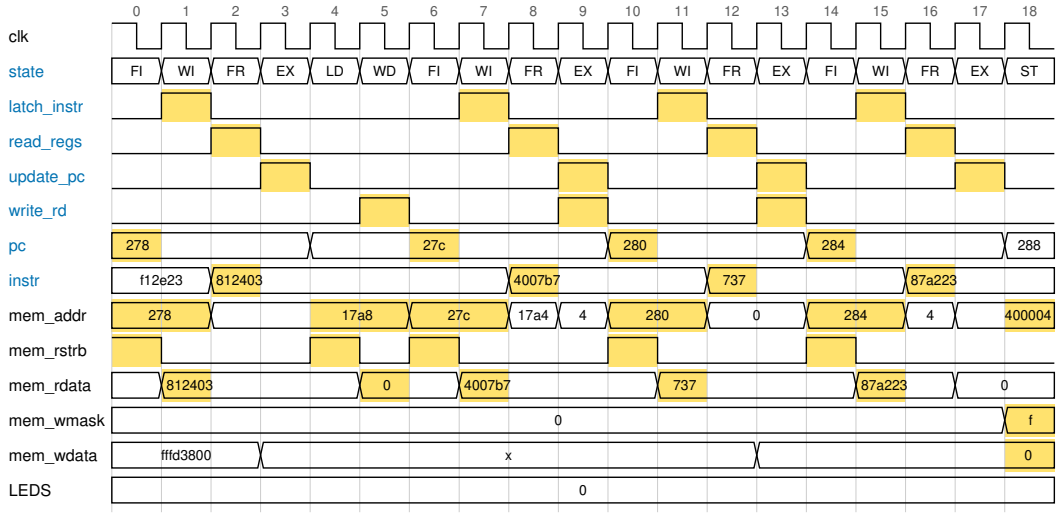


Figure 6.3: Four instructions of the mandelbrot demo in the simulated SoC: `lw s0,8(sp)` at `0x278`, `lui a5,0x400`, `lui a4,0x0` and `sw s0,4(a5)`, the store to the LED register. The core's signals are in blue, the bus in black. Bus values are in hex with leading zeros dropped. In each state the values to read, those the state uses or produces, are marked in yellow; an unmarked value is one nothing uses in that cycle, as `mem_addr` is in `FETCH_REGS` and `EXECUTE`. `mem_wdata` reads `x` while the instruction's `rs2` field names a register no instruction has written yet: the register file has no reset.

7 Memory and memory-mapped IO

To the core the three regions of the bus differ only in timing. The RAM and the IO registers answer every read in the next cycle and take every write on the edge it is presented, while the flash answers a read about 380 cycles later, and the core waits.

7.1 The RAM

`Memory` is 1536 words, 6 KB, of block RAM: two of the Kintex-7's 445 tiles. Its one port serves instruction fetch, loads and stores alike, so in BRAM boot the program, its data and its stack share it, the stack growing down from the top. The read is registered: the word addressed in the cycle of `ram_rstrb` appears on `ram_rdata` at the next edge, which is the one cycle every wait state of the FSM spends. A write needs no strobe and no wait: each of the four `ram_wmask` bits writes its byte lane on the edge where it is high, so `SB` and `SH` leave the rest of the word as it was (`notes.md`, "Per-byte write enables"). `IOBus` masks both strobes off outside RAM space, so a store to an IO register never also writes the RAM word with the same index (`notes.md`, "RAM strobes masked outside RAM space").

The RAM's contents at configuration are the firmware image, `rtl/firmware.hex`, read with `$readmemh` when the bitstream is synthesised, so a BRAM-boot program is part of the bitstream and a new program is a new bitstream. The file is read by bare name, relative to the working directory of the tool, and a missing file leaves the RAM undefined with no error (`notes.md`, "Bare file name in $readmemh").

7.2 The LED register and the UART

The IO registers are three addresses in the IO region, and none of them is memory: a store to one is an action and a load from one is a question.

LEDs. A store to `IO_BASE + 4` sets the LED register to the low byte of the store data. `SW` and `SB` both work, since `IOBus` ignores the byte mask. The register drives `LD0` to `LD7` directly, clears on reset, and reads as zero (`notes.md`, "LED register: reset not gated by the clock enable").

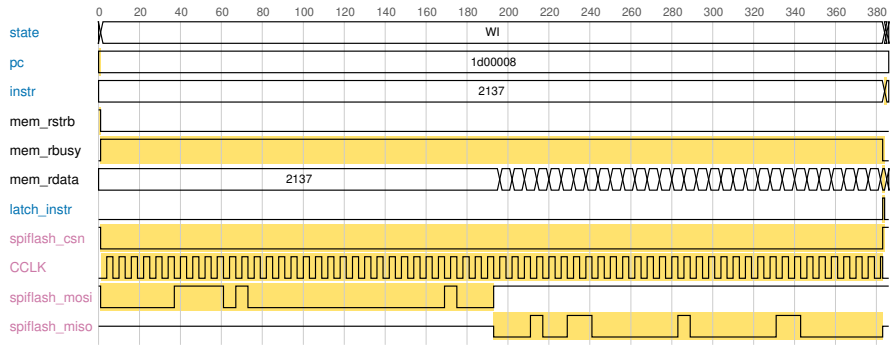
UART data. A store to `IO_BASE + 8` turns into `uart_valid`, a one-cycle pulse, with the low byte as `uart_data`. `UartTx` drops `ready` on the same edge and sends the byte as one start bit, eight data bits LSB first and one stop bit, each 1736 cycles long: 115 207 baud, 0.006 % above 115 200. A byte takes 17 360 cycles, 86.8 µs. The transmitter has no FIFO and ignores `valid` while it sends, so a byte stored too early is lost without a trace (`notes.md`, "UART send pulse and the status word (bit 9 convention)").

UART status. A load from `IO_BASE + 0x10` returns a word that is zero but for bit 9, which is `!ready`: the transmitter is busy. It is the only IO address that answers a read, and bit 9 is a convention between `IOBus` and the firmware, held equal by the memory-map lint rule. `fw/putchar.S` is the whole driver: it stores the byte, then loads the status word until bit 9 is clear, which is why the demos run at the pace of the UART.

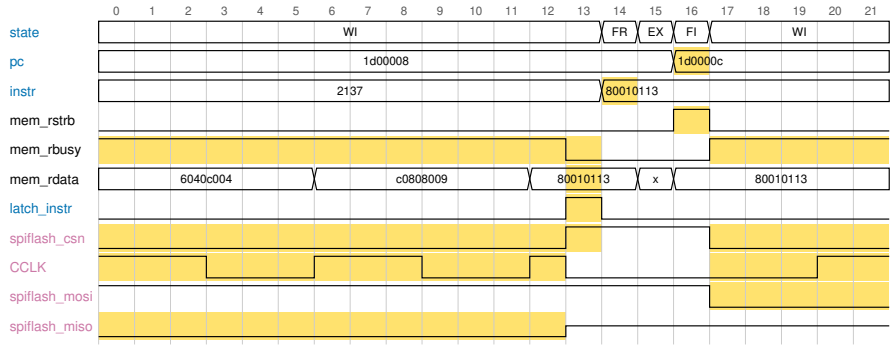
7.3 The flash stall

A read in flash space is the one access the core waits for. In the cycle of the read strobe, `IOBus` passes it to `SPIFlash` with the word address, and the controller drops chip select and

raises `mem_rbusy` at the next edge, and holds both for the whole SPI transaction (`notes.md`, "Only flash reads stall the core, read mux"). The FSM's two wait states hold while `mem_rbusy` is high (`notes.md`, "Instruction fetch stalls on mem_rbusy"), and `write_rd` in `WAIT_DATA` is gated by it, so a stalled load writes nothing until its word has arrived (`notes.md`, "Load stalls on mem_rbusy"). Figure 7.1 is one such stall, simulated: the third instruction of flash-boot start-up, fetched from `0x1D00008`.



(a) The whole instruction, 386 cycles: `FETCH_INSTR` at cycle 0, `WAIT_INSTR` for 383 cycles, then `FETCH_REGS` and `EXECUTE`.



(b) The end of the same transaction, 22 cycles from the last two data bits: chip select rises, `mem_rbusy` falls, the word is latched, and the next fetch starts the next transaction.

Figure 7.1: An instruction fetched from the flash, in the simulated SoC in flash boot (with a model of the flash's read command, booting mandelbrot's flash build). The core's signals are in blue, the flash controller's pins in purple. In each state, the values worth reading are marked in yellow. The unmarked ones are not used in that state and may hold garbage, so they can be ignored. `spiflash_miso` is drawn mid-way where the flash leaves it undriven: before its first data bit and after chip select rises. The transaction is 64 SPI clocks at a sixth of the system clock, 33.3 MHz: the Read command's opcode, `03h`, and the 24-bit byte address on `spiflash_mosi`, then 32 data bits on `spiflash_miso`. That is 382 cycles of `mem_rbusy`, and the instruction takes 386 cycles, 1.93 µs, where one fetched from the RAM takes 4. The word arrives lowest address first, each byte MSB first, as `13 01 01 80`, and the controller swaps the bytes into `0x80010113`, `addi sp,sp,-2048`, little-endian as the RAM returns it (`notes.md`, "Byte order of the received word"). Chip select rises right after the last bit, and `mem_rbusy` with it. In that cycle `WAIT_INSTR` latches the word and the core moves on. The controller's clock and its divider are held while chip select is high, so the first clock pulse of a transaction is as long as the others (`notes.md`, "Divider held in reset while idle (short first pulse)"). There is no cache and no prefetch: every word read from the flash is a transaction of its own, so code runs from the flash about a hundred times slower than from the RAM, which is what shapes flash boot in the next chapter.

8 Boot modes and the SPI flash

The core starts at its reset vector, the `SOC` parameter `BOOT_ADDR`, and that one parameter is the boot mode. At address 0 the core runs a program held in the RAM, BRAM boot. At `0x01D0_0000` it runs a program stored in the flash, in place, flash boot. The hardware is otherwise the same (`notes.md`, "Why the reset vector is a parameter, and why a jump at address 0 is not enough"). Table 8.1 compares the two.

	BRAM boot	Flash boot
Reset vector	`0x0000_0000`	`0x01D0_0000`, the default
Program held in	the RAM, from `rtl/firmware.hex`	the flash's code window
Room for code	6 KB, shared with data and stack	3 MB; data and stack in the RAM
A new program needs	a new bitstream	a flash write; the bitstream stays
Start-up code	`fw/start.S`	`fw/start_spiflash.S`
Speed	4 to 6 cycles an instruction	382 cycles more for each word read from the flash
Simulated	yes: the golden trace	the start-up only (Figure 7.1)

Table 8.1: The two boot modes. `make -C fw` builds either (`TARGET=bram` or `TARGET=flash`), and the README's "Running the demos" gives the steps on the board.

BRAM boot is the simple mode, and the one the regression runs: the program is part of the bitstream and runs at full speed, but it must fit in 6 KB. Five of the six demos do. Flash boot is for a program that does not fit, such as tinyraytracer, 29 KB. Its bitstream is built once, with the default `BOOT_ADDR`, and runs whatever program is written to the code window. A BRAM-boot bitstream sets `BOOT_ADDR` to 0 at synthesis.

8.1 The flash layout

The board's flash is a 32 MB S25FL256S. The core sees only its lower 16 MB, because three limits meet at 24 address bits, 2^{24} bytes: the flash's Read command, `03h`, carries a 3-byte address, which the chip completes into its lower half; the controller fills that field with a 22-bit word address and two zero bits; and in the memory map, bit 24 selects flash space, so the offset is the 24 bits below it (Section 4.2). Everything the design keeps in the flash fits in that half, so the upper 16 MB stays unused (`notes.md`, "Why the core reaches only the lower 16 MB of the flash"). The byte at chip offset o is at core address `0x0100_0000` + o. Figure 8.1 draws those 16 MB to scale, with the chip offsets under the bar and the core addresses over it (`notes.md`, "Flash memory map and boot address").

What the figure shows. The bar is the chip from offset 0 on the left to 16 MB on the right, in four regions:

- *Room for a power-on bitstream*, grey: offsets `0x000000` to `0xAE9E0F`, 10.9 MB. Nothing in this project writes there.
- *Free*: up to `0xBFFFFF`, unused by this design.
- *ST_NICCC's scene data*, green: 640 KB from `0xC00000`.

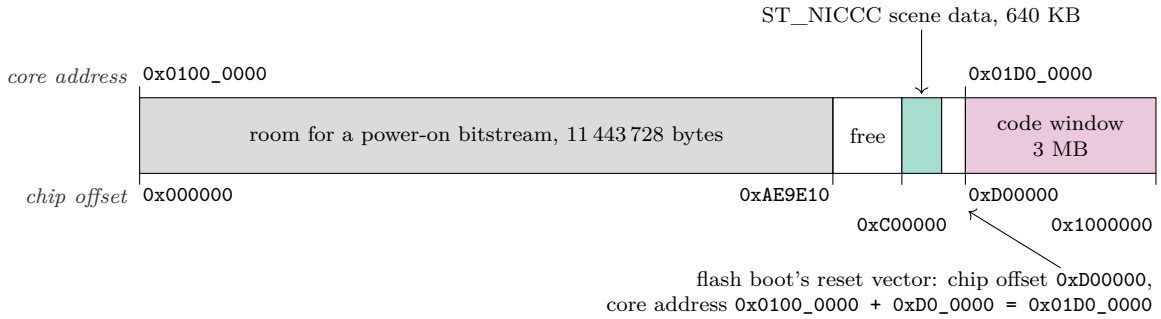


Figure 8.1: The flash layout, to scale: the lower 16 MB of the 32 MB chip, the part the core can reach. Under the bar, each byte's offset from the start of the chip; over it, the same byte's core address, `0x0100_0000` more. Both grow to the right. On my board the grey region holds Digilent's factory demo, whose data also reaches further, up to `0xDEFFFF`, so a write to the scene data or the code window overwrites part of that data.

- *The code window*, pink: the last 3 MB, from `0xD00000`, where a flash-boot program is written. Its first byte is flash boot's reset vector, core address `0x0100_0000` + `0xD0_0000` = `0x01D0_0000`. The arrow points at chip offset `0xD00000`, left of `0x1000000`, because the scale under the bar is the chip offset; the core address of the same point is the scale over the bar.

Each region starts on a 64 KB boundary, because the flash erases in 64 KB sectors, so one region can be rewritten without touching the others.

A power-on bitstream. The FPGA holds its configuration in volatile memory, so it forgets the design when the power goes. A bitstream reaches it in one of two ways. Over JTAG, Vivado's Hardware Manager sends it straight into the FPGA, where it lasts until the power goes; the flash is not touched, and every bitstream of this project is loaded this way. At power-on, with the mode jumper on QSPI, the FPGA reads a bitstream from the flash at offset 0 by itself, before any design runs: that is a *power-on bitstream*, which brings the board up with a design and no PC. A bitstream configures the whole FPGA, every LUT, flip-flop, routing switch and block RAM, used or not, so its size depends on the device, not on the design: every uncompressed bitstream for the XC7K325T is 11 443 728 bytes, this small SoC's included. The layout keeps that much room at offset 0, so that a bitstream of this design could be stored there. A flash-boot program is not part of the bitstream: it sits in the code window, where tinyraytracer takes 29 KB of the 3 MB.

This project does not store a power-on bitstream, for three reasons. Loading over JTAG takes about 7 s and touches nothing, while a power-on bitstream means erasing and writing 11 MB of flash for every change, and in BRAM boot every new program is a new bitstream. The board is never used without the PC anyway: the demos print to a terminal on the PC, over the UART. And offset 0 holds Digilent's factory demo, which this project keeps, so that the board stays as it was shipped: every flash write is checked first to hold no byte below `0xC00000`, so no write can replace the bitstream the board boots, and the one write that cleared the Quad Enable bit showed how easily a flash write can stop the board from configuring at power-on (`notes.md`, "Writing the flash clears the Quad Enable bit").

Digilent's factory demo. A Genesys 2 leaves the factory with a power-on bitstream of Digilent's already at offset 0, the factory demo: a design that shows the board working, with text on its OLED display (Figure 1 shows it running). This project never writes offset 0, so on my board the grey region still holds the factory demo, and the board boots it whenever it is switched on. The demo keeps data beyond its bitstream, from `0x600000` to `0x69FFFF` and from

`0x800000` to `0xDEFFFF`, so writing the scene data or a program overwrites part of that data, but never the bitstream: a readback of the whole chip after tinyraytracer was written differed from the one before only in the image's own bytes. The board still boots the factory demo at power-on, provided the write keeps the flash's Quad Enable bit (`notes.md`, "Writing the flash clears the Quad Enable bit").

From power-on to this design. Two bitstreams reach the FPGA, one after the other, and the second replaces the first:

1. *Power-on.* The configuration memory starts empty. With JP5 on QSPI, the FPGA's configuration logic reads the bitstream at offset 0, the factory demo, DONE lights and the demo runs. Nothing waits for JTAG: the demo runs until something else happens.
2. *This design, over JTAG.* Vivado first clears the whole configuration memory, as a press of the PROG button does, so the demo stops existing inside the FPGA. It then sends the whole bitstream, in about 7 s, which sets every LUT, flip-flop, routing switch and block RAM, used or not. Startup runs again, DONE lights, and the core starts at its reset vector, every flip-flop and block RAM at its initial value. The new bitstream replaces the old one and keeps nothing of it, which is also why every bitstream has the same size.
3. *Power-off.* The design is gone, and at the next power-on the factory demo boots again. With JP5 on JTAG instead, the FPGA configures nothing at power-on and waits, DONE dark, for a bitstream over JTAG.

Loading a bitstream over JTAG never reads or writes the flash: the demo's bitstream at offset 0 stays as it is, and in flash boot the design itself only reads its code window. JTAG reaches the flash only when Vivado is asked to write it, with *Program Configuration Memory Device*. Vivado then first loads a helper bitstream of its own into the FPGA, which connects the JTAG cable to the flash's pins, and writes through it only the bytes the image covers. That helper too replaces whatever design ran, so this design's bitstream is loaded after the write, and no write of this project reaches below `0xC00000`.

Flash boot is not booting from offset 0. The flash holds two different things. At offset 0 is the FPGA's configuration, the bitstream, read once at power-on. In the code window is the core's program, which `SPIFlash` fetches word by word while the design runs. The boot modes of this chapter are about the program only: a BRAM-boot bitstream and a flash-boot bitstream alike can be loaded over JTAG or stored at offset 0. tinyraytracer, too large for the RAM, runs in flash boot on the board (Figure 1.3). The other demos run in BRAM boot, ST_NICCC among them, though it reads its scene data from the flash.

8.2 CCLK and STARTUPE2

The flash has four pins. Chip select and the two data lines become ordinary FPGA pins once configuration ends, so they are plain ports in the constraints file. The clock does not: it is CCLK, a dedicated configuration pin, which a design can drive only through the `STARTUPE2` primitive [8]. `SOC` wires `SPIFlash`'s clock to its `USRCCLKO` input and takes back its `EOS` output, end of startup. `IOBus` passes a flash read to the controller only once `EOS` is high (`notes.md`, "SPI flash CCLK access through STARTUPE2").

The hand-over costs the first transaction. Vivado's model of `STARTUPE2` holds CCLK high after end of startup until `USRCCLKO` has risen twice, so the first transaction loses two of its 64 clock edges and the flash misreads its command. In simulation this loses the first fetch of `fw/start_spiflash.S`, and it is the likely reason flash boot needs one press of the reset button after each load on the board (`notes.md`, "Flash boot loses its first fetch").

8.3 Execute-in-place

In flash boot the program runs where it is stored: the core fetches every instruction straight from the flash, so the code, `.text`, and the constants, `.rodata`, are never copied to the RAM. A store cannot write the flash, though, so every variable must live in the RAM. `fw/spiflash.ld` places the initialised variables, `.data`, and the ones that start at zero, `.bss`, in the RAM, and keeps the initial values of `.data` in the flash, right after the code. Before `main` runs, `fw/start_spiflash.S` sets `gp` and `sp`, fills `.bss` with zeros and copies the initial values of `.data` from the flash into the RAM.

The same copy moves the code that runs most into the RAM, where it runs about a hundred times faster: `libgcc`'s multiply and divide, which every `*`, `/` and `%` in C calls on RV32I, `fw/putchar.S` and `fw/print.c`, which every character passes through, and any function marked `.fastcode`. The code in the flash calls them at their RAM addresses. In mandelbrot's flash build, 1 228 bytes are copied and 1 172 stay in the flash. The copied code shares the 6 KB of RAM with the data and the stack, and nothing checks that the stack stays clear of it (`notes.md`, "firmware_words arguments, -max_addr as a stack guard").

9 Verification

The design is checked at three levels: one module against values worked out by hand, the whole SoC against a recorded trace, and the repository against mechanical rules. Each check is a make target, a *gate*, listed in Table 9.1.

Gate	What it runs	Time
`make lint`	the repository's mechanical rules: file and signal names, the firmware's copies of the memory map, tool by-products beside the sources	seconds
`make unit`	the ALU bench (Section 9.1)	seconds
`make smoke`	`unit`, the firmware check, then the start of the golden trace	25 s
`make check`	`unit`, the firmware check, the whole golden trace, and synthesis	minutes
`make baseline`	synthesis and implementation, with the timing reports of Chapter 10	about a minute

Table 9.1: The main gates, targets of the root `Makefile`, whose header lists them all. The firmware check holds the firmware's copies of the memory map to `rtl/defines.vh`, and rebuilds `rtl/firmware.hex` and fails unless it comes out byte for byte the same. Each gate but `check`, which only runs others, writes a log in `build/logs/` that ends in `PASS` or `FAIL`.

9.1 The ALU unit bench

`sim/ALUBench.v` drives `rtl/ALU.v` directly and checks ten right shifts, `SRL`, `SRA`, `SRLI` and `SRAI`, against results worked out by hand from the RV32I specification [2]. The operands are negative, positive and most-negative, shifted by 0, 4, 9 and 31 places, with the upper bits of the shift amount set, which the ALU must ignore. The operand the instruction does not use is `0xDEADBEEF`, so a wrong operand choice shows.

9.2 The golden trace

`sim/bench.v` runs `SOC` in BRAM boot and records only what can be seen from outside the chip: each byte on the UART line, decoded from the pin, and each change of the LEDs, with its time. It reads no internal signal, so the RTL can be restructured and the same trace still applies. It writes one line per event and an `END` line:

```
UART 1b
  ⋮
LEDS 1031464038 01
END stop_leds cycles=206292808 uart_bytes=11014 leds=01
```

The run stops when the LEDs become 1, which mandelbrot writes at the start of its second frame, so the trace holds exactly one frame: 206 292 808 cycles, 1.031 s of simulated time and 11 014 UART bytes (`docs/baseline.md`). The `END` line carries the cycle count, so a change that keeps every byte but makes one instruction a cycle slower fails too. The trace's UART bytes are, byte for byte, the frame `fw/mandelbrot.c` prints when compiled natively, and the frame the board sent (Section 1.1).

9.3 The regression

`sim/run_regression.sh` simulates `SOC` and the bench in xsim and passes only when the new trace is byte-identical to the golden one. The full run takes about six minutes. With `--max-cycles` it is the smoke run, about 25 s, whose shorter trace must be the start of the golden one. It runs in its own build directory, since `Memory.v` reads the image by bare name (`notes.md`, "Bare file name in $readmemh"). `make golden` records a new golden trace, only for an intended change of behaviour, whose commit says why.

The regression has two gaps. It runs one program: mandelbrot uses 24 of the 37 RV32I instructions other than `FENCE`, `ECALL` and `EBREAK`, and never executes `SLL`, `SRA`, `SLT`, `SLTU`, `XOR`, `SLTI`, `SLTIU`, `XORI`, `ORI`, `LB`, `LH`, `LHU` or `SH`. And it runs one boot mode: the bench has no flash model.

10 Implementation results

Every number in this chapter is from the routed design that `vivado/baseline.tcl` builds, non-project, for the XC7K325T-2FFG900 with the constraints file's 200 MHz clock, in Vivado 2025.2.

10.1 Utilization

The whole SoC is 984 LUTs and 244 flip-flops, under half a percent of the device, and two of its 445 block RAM tiles. Table 10.1 gives the split per instance.

Instance	LUTs	as RAM	FFs	BRAM tiles
SOC, the whole design	984	44	244	2
`Processor`, the core	941	44	130	0
its own logic: pc, instruction, address adders	89	0	63	0
`Control`, the FSM	58	0	3	0
`ALU`	2	0	0	0
`RegisterFile`	797	44	64	0
`Memory`, the RAM	0	0	0	2
`IOBus`, the LED register	0	0	8	0
`UartTx`	25	0	26	0
`SPIFlash`	19	0	80	0

Table 10.1: Utilization of the routed design, per instance. Each row counts where the cells ended up after synthesis, not where their Verilog is. A parent's LUTs can be fewer than its children's sum, since a LUT shared across the hierarchy counts once in the parent. The device holds 203 800 LUTs, 407 600 flip-flops and 445 block RAM tiles, and the design uses no DSP blocks.

The core is nearly all of the logic, and inside it the rows follow the netlist rather than the sources: synthesis optimises across module boundaries, and the ALU's adder, shifter and comparators end up in cells under `cpu/register_file`, leaving 2 LUTs in `ALU`. That is also why the RTL schematic of Figure 6.2 comes from the elaborated design. Of the register file's own LUTs, 44 are distributed RAM: the 32 registers, one copy for each read port. The peripherals are small: `SPIFlash` is mostly flip-flops, its shift registers and counters, `UartTx` is a baud counter and a shift register, `IOBus` is the 8 flip-flops of the LED register, and `Memory` is the two block RAM tiles alone.

10.2 Timing

The one constraint, the 5 ns period of the 200 MHz clock, is met on every one of the 1229 timed endpoints: the worst setup slack (WNS) is 0.219 ns and the worst hold slack 0.113 ns, so the design would close at up to 209.2 MHz. The worst path (Figure 10.1) is the load path.

It starts at `cpu/instr_reg[3]`, an opcode bit of the instruction register, and ends at the data input of the register file's LUT RAM: the write of `rd`. Its first 3 LUTs decode the instruction and choose its immediate, a carry chain of 7 `CARRY4` adds it to `rs1`, the load-store address, as far as bit 24, the bit that selects the flash region, and the last 5 LUTs use that bit to choose the bus's read data, cut out the loaded byte or halfword and pass it to the write port: 15 levels in all. The address is not registered, so the write in `WAIT_DATA` waits on the whole adder again. Of the path's 4.312 ns, 1.291 ns are in the cells. The other 3.021 ns, 70 %, are in the routing between them. Routing dominates, as it usually does on an FPGA. Fewer levels, not faster cells,

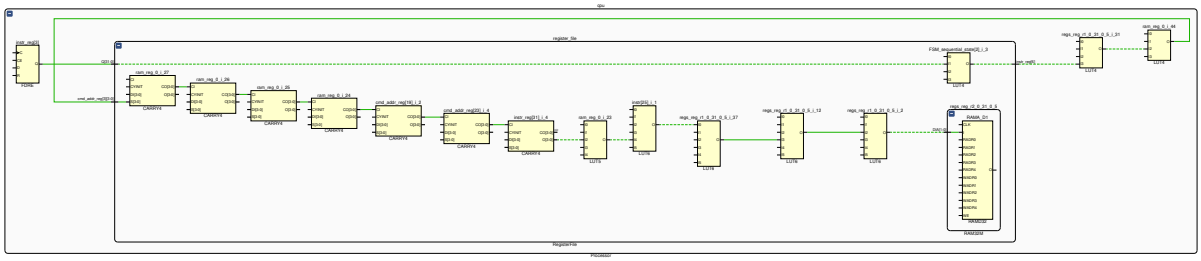
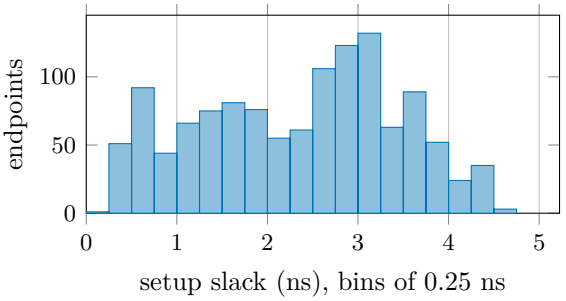


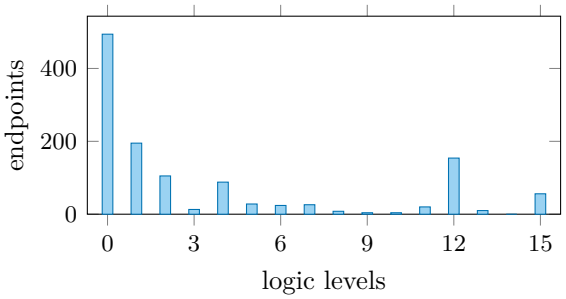
Figure 10.1: The worst setup path in Vivado's schematic, from `instr_reg[3]` at the left to the register file's write data at the right, inside the boxes of `cpu` and `cpu/register_file`: 7 `CARRY4`, 4 `LUT6`, 3 `LUT4`, 1 `LUT5`, between a flip-flop and a LUT RAM.

would shorten the path. Registering the load-store address, or only its region bit, would take the adder off it (`notes.md`, "The worst path ends in the register-file write").

Figure 10.2 shows the margin of the rest of the design: most endpoints have well over a nanosecond to spare, and 494 have no logic at all between their two ends.



(a) Setup slack of each endpoint's worst path. 188 endpoints, 15 %, have less than 1 ns, and the most has 4.516 ns.



(b) Logic levels of the same paths. 56 endpoints sit at the deepest, 15 levels.

Figure 10.2: All 1229 timed endpoints.

An instruction on the path. The parts of the path do not change in the same cycle. `instr` is latched at the end of `WAIT_INSTR` and `rs1` at the end of `FETCH_REGS`, so the address is settled from `EXECUTE` on. In `WAIT_DATA`, the cycle that writes `rd`, only the RAM's word, read in `LOAD`, is new. Vivado does not know the order of the states: with no multicycle constraint, it assumes that any flip-flop can change at any edge, and times the whole path, from `instr_reg[3]` to the write, within one 5 ns cycle.

That assumption, the default of static timing analysis, holds for only part of this path. It holds for the last part: the RAM's word appears at the end of `LOAD` and is written at the end of `WAIT_DATA`, one cycle later. It does not hold for the rest: `instr` changes at least two cycles before any write to `rd`, and `rs1` three cycles before a load's write, so in the running design the decode and the adder have several cycles to settle, not one. The assumption is therefore pessimistic, never optimistic. A multicycle constraint could tell Vivado so, but the design meets 5 ns without one, and a wrong one would hide a real failure. The reported slack is a bound that the design meets, not a race that every load runs.

10.3 Power

Vivado estimates 194 mW on the chip (Figure 10.3). 160 mW of it is static: the leakage of the whole Kintex-7, which a configured FPGA draws whatever the design. The design's dynamic power is 35 mW, from 5 to 9 mW each for the block RAMs, the signals, the logic, the I/O and the clock tree. The estimate assumes Vivado's default toggle rates.

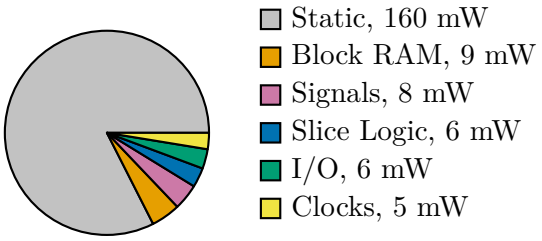
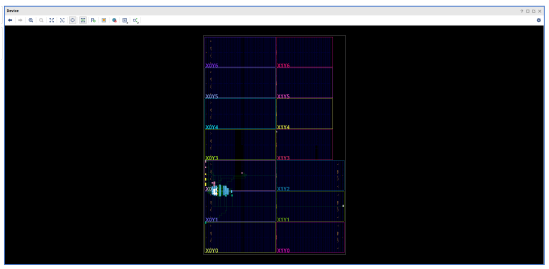


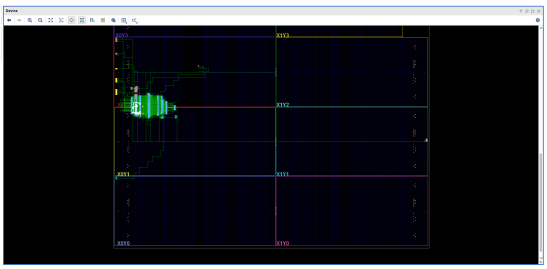
Figure 10.3: On-chip power, 194 mW in all, from `report_power`: a vectorless estimate at Vivado's default toggle rates, with a junction temperature of 25.3 °C.

10.4 The design on the device

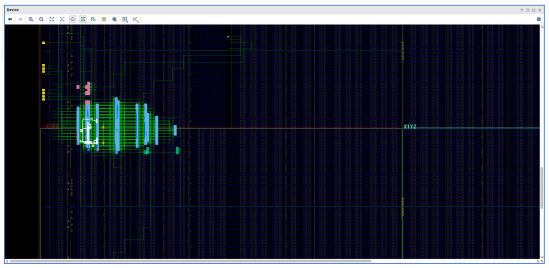
Figure 10.4 is Vivado's Device view of the same routed design, with its worst path. The whole SoC is a small patch at the left edge of the lower half of the device, and the worst path stays inside it.



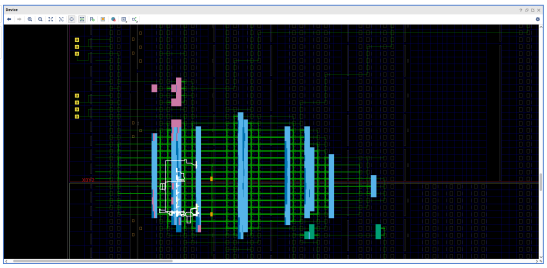
(a) The whole device, its 14 clock regions outlined. The design is the coloured patch at the left edge of the lower half.



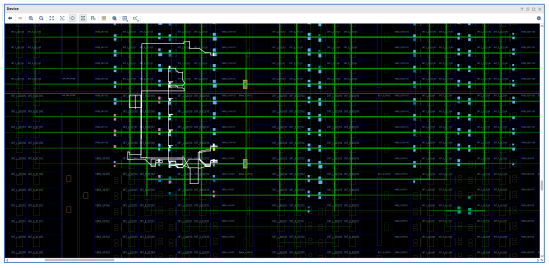
(b) The lower clock regions. Vivado draws the routed nets in green, and some run from the design out to the device's edges.



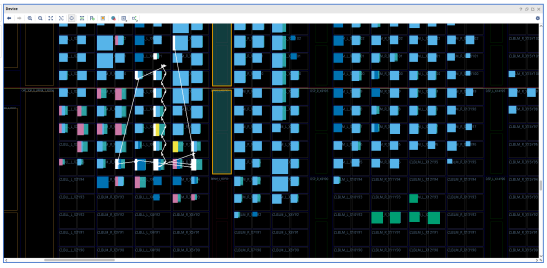
(c) The design: the register file and the rest of the core in the middle, the LED outputs at the left edge, and the worst path in white.



(d) Around the worst path, which ends at a write port of the register file's LUT RAM.



(e) The worst path with Routing Resources on: each net follows the wires the router chose through the interconnect tiles.



(f) Routing Resources off, zoomed further in: each net is a straight line from its driver to its load, over the slices that hold the path's cells.

Figure 10.4: Vivado's Device view of the routed design, zoomed in from the whole device to the worst setup path, `cpu/instr_reg[3]` to the register-file write. The modules are in their colours and the worst path is selected, in white.

11 For the next student

This chapter lists ideas for extending the core and the system, ranked by what each gains for the work it takes. None is begun.

1. **Pipelining:** overlap the fetch of one instruction with the execution of the last, to retire close to one instruction a cycle.
2. **The M extension:** multiply and divide in hardware, on the device's DSP blocks, instead of calls into `libgcc`.
3. **Inputs from the PC:** today the PC only listens. A UART receiver on the unused `uart_tx_in` pin, with a data register and a "byte received" status bit in the IO region, would bring the keys typed in the PC's terminal to the program, and the board's five push buttons and eight switches could be read the same way. With input the system can run interactive programs, games among them, and load new programs over the cable, a serial boot loader. A game such as Doom is the far goal: it also needs megabytes of RAM for its code and data, which the DDR3 item below gives, a screen, which the HDMI item gives, and the speed that pipelining and the M extension bring.
4. **Interrupts:** the control and status registers of Zicsr and a trap state, with a timer and the UART as the first sources.
5. **DDR3:** the board's 1 GB as main memory, through AMD's Memory Interface Generator and behind a cache.
6. **HDMI:** a video output, from a frame buffer in DDR3.

12 Conclusions

The project set out to run compiled C on a processor of its own, on a real board, and to understand every layer in between. It did. A multi-cycle RV32I core of 984 LUTs, with 6 KB of RAM, three IO registers and a flash controller, runs at the Genesys 2's 200 MHz with 0.219 ns to spare, draws the Mandelbrot set over the UART from the block RAM, and renders a ray-traced picture in place from the flash. Its educational value is the path more than the core: a line of C passes the compiler, the linker, the firmware image, synthesis, the bitstream and the bus before it lights an LED, and building each stage by hand turned layers that are usually black boxes into parts that can be read, changed and checked.

That path is also where I learned the most. Before this project a processor was a diagram in a course. Now each of its parts is something I have written, simulated, measured and watched run on a board, and each layer taught something no single course covers:

- **The instruction set.** Decoding RV32I's formats, immediates and calling convention by hand showed why the ISA is built as it is, and how a compiler maps C onto it, down to a multiplication that becomes a call into `libgcc`.
- **The microarchitecture.** The FSM follows from the memory: a RAM that answers a read one cycle late and writes on the same edge decided the states, and a timing report showed what an unregistered address costs.
- **The system around the core.** A bus, an address decode, memory-mapped IO and a stall for a slow device are what turn a core into a computer, and the firmware's start-up code and linker scripts are what let C run on it from the RAM or in place from the flash.
- **The FPGA and the board.** Pins, banks and I/O standards, a differential clock, the configuration logic that owns the flash's clock, and the way a bitstream reaches the FPGA, over JTAG or from the flash, are what a design must get right before it runs at all.
- **The FPGA and Vivado.** I learned what an FPGA is: not a processor that runs a program, but a grid of configurable logic, LUTs that compute any function of six inputs, flip-flops, carry chains, block RAMs and DSP blocks, joined by a programmable routing network, all of it set by the bitstream. Vivado, AMD's tool, turns Verilog into that bitstream, and I learned to work with it in both ways: in its GUI, a project with sources, constraints, simulation, schematics and the Hardware Manager, and in batch mode from Tcl scripts, which make every build repeatable. I learned what each step does on the chip. Synthesis turns the RTL into a netlist of the device's own cells: the RAM became two block RAMs, the register file LUT RAM and the adders carry chains, and module boundaries dissolve as logic is optimised across them. Placement gives each cell a site on the chip, and routing joins the sites through the interconnect, which on an FPGA is most of the delay: 70 % of the worst path is wire. Static timing analysis then checks every path against the clock constraint. Reading its reports, the schematics and the Device view is how I learned what my Verilog had become in silicon.

Building the whole system alone, from the first lit LED to programs larger than its RAM, gave me a working understanding of a computer from the FPGA's fabric up to the C source, and the confidence to extend it: Chapter 11 lists where to go next.

Bibliography

[1] B. Levy, *learn-fpga*, "From Blinker to RISC-V", GitHub repository. `https://github.com/BrunoLevy/learn-fpga`

[2] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, document version 20191213, RISC-V Foundation.

[3] Digilent, *Genesys 2 Reference Manual*. `https://digilent.com/reference/programmable-logic/genesys-2/reference-manual`

[4] Digilent, *Genesys 2 Schematic*, revision H.1, 22 pages, 2015, in the Genesys 2 Resource Center. `https://digilent.com/reference/programmable-logic/genesys-2/start`

[5] D. Sokolov, *tinyraytracer*, "Understandable RayTracing in 256 lines of bare C++", GitHub repository. `https://github.com/ssloy/tinyraytracer`

[6] Oxygene, *ST-NICCC 2000*, Atari ST demo, and its polygon stream as published for the ST-NICCC competition. `http://arsantica-online.com/st-niccc-competition/`

[7] FTDI, *FT232R USB UART IC Datasheet*.

[8] AMD Xilinx, *7 Series FPGAs Configuration User Guide*, UG470.

[9] AMD Xilinx, *7 Series FPGAs SelectIO Resources User Guide*, UG471.